Stochastic Summaries: Characterizing Structured Datasets Using Probabilistic Programs

JACK STANEK, University of Wisconsin – Madison, USA
ADITHYA MURALI, University of Wisconsin – Madison, USA

What does my data look like? We study the problem of representing and learning *summaries* of datasets that account for both their structural and statistical properties. This problem is widely useful across many application domains that involve large datasets and analysis/modeling of data, and is of particular interest to Programming Languages. In particular, in this work we focus on the problem of learning these "stochastic summaries" for tree-like data composed of inductive datatypes and argue that probabilistic programs over inductive datatypes can effectively represent the desired summaries. We present Tali, a Probabilistic Programming Language over inductive datatypes and study the problem of learning such programs from data samples. We identify a semantic property, called *Observation-Bounded Compilability*, which identifies a set of recursive probabilistic programs which admit exact inference, and can represent many useful and complex probabilistic structures. We then propose an algorithm for learning Tali programs satisfying the OBC property from observations using a combination of program synthesis (stochastic search) and maximum likelihood inference. We implement our learning framework, evaluate it on a diverse benchmark suite, and record our observations on the merits of our approach. We demonstrate the utility of learning stochastic summaries as symbolically-represented priors for downstream learning tasks. We also showcase another utility by using our technique to learn operational profiles of real-world datasets.

CCS Concepts: • **Computing methodologies** → **Unsupervised learning**; *Machine learning approaches*; • **Mathematics of computing** → *Probabilistic representations*; • **Theory of computation** → **Probabilistic computation**.

1 INTRODUCTION

What does my data look like?

Data distributions are a ubiquitous construct in computer science. The most well known example of this is in machine learning, where the formal objective of learning problems is to obtain the natural "data generating distribution".

The representation, modeling, and analysis of large data sets is itself an important research question that powers several applications [49]. One view of the goal of such efforts in modeling data distributions is to construct *summaries* or abstractions for them. Empirical datasets can be thought of as samples from an underlying "true" generative model. In contrast to the more commonly studied logical or language-based abstraction mechanisms which capture structural properties or invariants, data distributions capture both statistical and logical/relational properties of the domain of interest. These distributional abstractions can also be compositional: probabilistic graphical models, causal models, neural networks, and probabilistic programs are all examples of compositional languages for representing data distributions.

Summaries for Data Distributions and Programming Languages. The representation and learning of probabilistic data summaries is also of specific interest to the field of Programming Languages. The design of abstraction spaces that admit expressive and compositional representations of practically useful data distributions while simultaneously being amenable to efficient learning is a goal of many research disciplines in PL, including program synthesis, probabilistic programming, and neurosymbolic programming/learning. This point has also been argued in several prior works

Authors' addresses: Jack Stanek, University of Wisconsin – Madison, Madison, WI, USA, jstanek@cs.wisc.edu; Adithya Murali, University of Wisconsin – Madison, Madison, WI, USA, adithyamurali@cs.wisc.edu.

in probabilistic programming, and in particular the work in [49] also pursues learning interpretable and operationalizable data summaries for time series data and tabular data.

Additionally, with stochasticity being increasingly woven into the vocabulary of modern system building and algorithm design [4, 14, 33], the study of stochastic data summaries can prove to be a useful tool. For example, prior literature has explored the concept of *operational profiling* [39], where one specifies a generative function that is meant to capture a program's usage patterns. Testing or analyzing systems against operational profiles (as opposed to the typical test suites obtained from analysis of the program's logic) can provide a useful perspective on the practical functionality and robustness of software [2].

Synthesis of Probabilistic Programs over Structured Data. We make two choices towards defining a formal problem. First, we consider distributions over structured data, specifically generalized labeled trees in the form of inductive datatypes. This choice is motivated by several factors. First, computing with inductive datatypes is supported by many algorithms, and more generally, labeled trees are a ubiquitous construct in software. Second, the structure of data can mirror information about its latent generative process, such as recursive patterns or local dependencies. By contrast, consider integer programs, which can perform arithmetic operations that collapse the individual values (and therefore their distributions) into aggregates; intermediate distributions must be marginalized, which may be intractible or require approximate methods. Finally, inductive datatypes allow us to study complex recursion patterns in generative processes that exploit repeated substructure. Inductive datatypes are in fact an under-explored area in the study of distributions over structured data, with contemporary literature focusing mostly on tabular data [1, 44, 55, 60].

The second choice we make is to represent distributions as probabilistic programs. This choice is also motivated by several considerations. First, probabilistic programs are a natural choice for representing generative processes. This perspective is itself not new; in fact, it is one of the central ideas behind probabilistic programming at large [23, 24, 57], and more specifically has also been heavily utilized by the literature on testing [10, 21, 22, 45, 56]. Second, we are inspired by literature on operational profiling [26, 39], where the skeleton or sketch of the distribution is determined at least in part from knowledge of the data domain. For example, a distribution over binary trees could utilize "knobs" relating the key stored at a node to whether it is a leaf, its height, or perhaps the keys stored at the children of the node. This framing also allows us to utilize paradigms from the literature on program synthesis where learning is modeled as the search for an optimal candidate in a given grammar [34, 42, 49]. Third, as discussed above, programs can easily (and explicitly) represent complex recursion patterns over tree-like data compared to other choices of representation. Such programmatic representations can be easily integrated into existing workflows where data profiles are needed. For example, one can use a program learned from examples to generate realistic test cases.

> *In this paper we formalize stochastic summaries as a class of probabilistic programs over inductive datatypes and develop techniques for learning them from sample data.*

Main Contributions

We advance the above vision in this work via the following contributions:

1. Observation-Bounded Compilability. The main contribution of this paper is the insight that productive recursion can be exploited to guarantee exact inference for recursive programs over inductive datatypes (Section 4.4). This choice is motivated by our learning domain of labeled trees. Productivity is the semantic property that every finite approximation of a possibly-infinite output (e.g., every finite prefix of a stream) is produced in finitely many steps [3]. In our setting, we wish to

learn a program which fits a set of observations. Conditionalizing a productive program's execution on the fact that a given output was generated, productivity allows us to "walk" the execution of the program in lock-step with structure of the observation, pruning dead execution paths which produce constructors that are not present in the observation.

We formalize this notion by defining the *Observation-Bounded Compilability* property, which identifies a class of programs for which a given observation directly bounds the depth of recursive calls, and thus the number of proabilistic choices, which are consistent with the production of that observation. Therefore, in contrast to statically bounded recursion, the OBC property picks out a *parametrically bounded* class of programs for which each observation corresponds with a straightforward unrolling of the program. This unrolling immediately enables both the exact inference algorithm noted in the previous paragraph, but also marginal likelihood techniques for program selection, as well as parameter tuning methods. For values produced by an unambiguous program trace, the observation likelihood factors into a product over Bernoulli random variables, and we can quickly compute marginal likelihoods in closed form over beta priors [17]. For ambiguous observations, we can instead use approximate variational inference techniques [5, 7] to compute lower bounds on the marginal likelihood. Finally, given a program, we can construct a finite-size arithmetic circuit representing the likelihood function given the symbolic probabilistic parameters. We can then use the circuit to tune parameters using maximum-likelihood estimation (MLE) via gradient-based methods.

2. Probabilistic Programs as Symbolic Priors for Inductive Data. Our second contribution is the use of probabilistic programs as *teacher distributions* for downstream learning tasks over inductive data. Over the past decade, the machine learning field has seen an explosion in the use of large, scalable models such as transformers [58]. By virtue of their capacity and generality, these models can learn very complex patterns, but this generality is a double-edged sword: they may need extraordinary amounts of data to generalize to unseen samples. Probabilistic programs, on the other hand, directly denote an exact distribution with particular properties and constraints. In settings where values have precisely defined logical properties or internal invariants, recursive probabilistic programs are a natural way to construct generative models.

To this end we develop a program synthesis technique which fits recursive probabilistic programs as generative models of real-world data. Our synthesis algorithm enumerates recursive program structures which support the training set, using parent annotation [29] and Bayesian model merging [28, 53] to identify local data dependencies; enumerated proposals are scored by marginal likelihood. Our method then uses a stochastic Metropolis-style search over the program space using subtree replacement to locate additional candidate programs, which are scored in similar fashion. During synthesis, we record the top-k candidate programs, and at the end of the process we fit parameters by maximum likelihood using gradient-based methods, returning the best-fit program.

We evaluate the effectiveness of our synthesis algorithm on real-world datasets from the software engineering literature [13, 37]. We execute our synthesis algorithm on these data, and evaluate the effectiveness of using synthesized programs as *symbolic priors* over the data. Our method uses the synthesized program as a generator of large training corpora; we are thus using the program synthesis step as a way to identify and amplify value constraints in the data, where a pretrained transformer with an uninformative pretraining prior would require a much larger training set to identify. We focus on the low-data regime, evaluating the method on 10-300 training samples. We find that pretraining on a synthetic corpus produces a model which fits the data better than the the program itself on average. Our method is also able to identify local and contextual invariants within structured data, such as ordering constraints.

```
define sampleSexpr(a : Atom) : Sexpr =
  if bern θ1 then
    SAtom(if bern θ2 then a else
      if bern θ3 then AtomA else AtomB)
  else if bern θ4 then
    Nil else
    Cons(sampleSexpr(next(a)), sampleSexpr(next(a)))
```

Fig. 1. Stochastic S-expression sampler. Atoms are sampled both at random and from a deterministic set of rotating atom choices. Each probabilistic choice contributes to some aspect of the return value's structure and content.

3. The Tali Probabilistic Programming Language. Our final contribution is the implementation of Tali , a simple first-order probabilistic programming language over inductive datatypes. We give a formal account of the syntax and semantics of our language in Section 4, and describe a type system, adapted from Atkey and McBride [3], which guarantees that well-typed programs exhibit the OBC property. Tali supports unbounded recursion, inductive datatypes, standard control flow constructs, and exact inference over observed samples. We also implement an copmiler which constructs arithmetic circuits representing observations' likelihood functions over symbolic parameters.

2 OVERVIEW

This work centers on the following high-level question: given a dataset of structured, inductively defined values, what would a compact and interpretable summary of that dataset look like? Consider the probabilistic program in Figure 1, which models a generative process that produces S-expressions according to a simple probabilistic grammar. It generates atom constructors both stochastically and from a rotating deterministic set of atoms. Each flip in the function body contributes to some aspect of the return value's structure or content: for instance, the results of bern θ_1 and bern θ_4 determine the type of S-expression that is generated, while bern θ_2 and bern θ_3 determine which atom is produced in the SAtom branch. For example, if next(AtomC) = AtomA, suppose we evaluate the call sampleSexpr(AtomC) with probabilistic choices bern θ_1 = false, bern θ_4 = false, bern θ_2 = true, and finally bern θ_1 = true on the recursive call. Such an execution would return:

Cons(SAtom(AtomA), SAtom(AtomA))

Despite its brevity, our stochastic sampler represents a nontrivial probability distribution with infinite support while remaining highly interpretable: its meaning can be understood simply by reading the program. Neural models over serialized trees could fit the same distribution, but their parameter weights would offer little insight into its recursive structure. The probabilistic program, by contrast, explicates this structure directly.

Probabilistic Programs as Hypothesis Spaces. The structure of data can often encode information about the underlying generative process. For example, in our S-expression generator, the program constructs an observation step-by-step, with each recursive call and probabilistic choice corresponding to a particular node in the expression tree. More complex tree structures may have more sophisticated internal constraints than this S-expression generator, but the principle remains the same: structure is a direct result of the probabilistic choices made by the program. Indeed, existing neural program learning systems such as DreamCoder [16] and earlier Bayesian learning techniques [34] frame the learning task explicitly as a structure discovery problem over data.

The space of recursive probabilistic programs is expressive, but it also includes programs with undesirable properties, such as nontermination or wasteful computation. For instance, our S-expression sampler could have been naïvely written to generate unrestricted random S-expressions, iterating until a particular condition is satisfied—for example, until certain ground atoms appear in a particular order. Such programming patterns can make probabilistic inference over observations difficult or impossible. In general, efficient inference is hampered by path explosion, marginal distributions without analytic closed forms, or undecidability. In order to avoid such pathological programs, a well-behaved subclass of programs is needed.

Efficiently Learnable Data Generators. The key observation we leverage in this paper is that, for certain well-behaved programs, sources of randomness can be mapped cleanly to aspects of the sampled values. Returning to our S-expression example, consider computing the likelihood of observing Cons(SAtom(AtomA), SAtom(AtomA)). The top-level call consumes two flips, bern θ_1 and bern θ_4, resolving the Cons branch. Each recursive sub-call unrolls one level further, consuming bern θ_1 and bern θ_2 to resolve the AtomA branches. The depth of unrolling is determined entirely by the structure of the observation itself: a value of depth two requires two levels of unrolling (though the lower level of sub-calls contains two independent calls). This is loosely analogous to backpropagation-through-time (BPTT) in the context of recurrent neural networks (RNNs) [59], where a bounded unrolling approximates inference by truncating recursion depth, trading exactness for tractability. Unlike BPTT, where truncation introduces approximation error, well-behaved programs need not sacrifice exactness: because the space of random choices is bounded by the observation, a truncated computation can remain exact. We formalize this insight as a property called *Observation-Bounded Compilability* in Section 4.3.

3 BACKGROUND AND PROBLEM DEFINITION

In this paper we use the convention that syntactic elements annotated with an overline (for instance, $\overline{e}$) refer to finite sequences of those elements. In inference rules, this notation indicates that the rule, premise, or conclusion is implicitly quantified over the elements of the sequence. A superscripted overline (e.g. $\overline{e}^n$) indicates that there are exactly n elements in the sequence.

3.1 Semantic domain

Our semantic domain is a set of finite labeled trees, which we take to be the *observable values*, meaning the objects that a dataset consists of, and consequently the objects against which our semantics assigns likelihoods. (The semantics of Section 4.2 works in a slightly larger set, generalizing the observable values to partially-observed values containing placeholders for parts of a value that have not been produced yet. The observable values reappear there as the maximal elements of this set.)

We fix a finite set of sorts $\mathcal{T}$ and a countable ranked alphabet of finitary constructor symbols C, forming a multi-sorted signature $(\mathcal{T}, C)$. We refer to the sorts of $\mathcal{T}$ as "types," and each constructor is associated with a unique type. We denote the set of constructors associated with type T as C_T, and require that each type has at least one associated constructor; that is,

$$\biguplus_{T \in \mathcal{T}} C_T = C.$$

The domain of values of type $T \in \mathcal{T}$ is the set of well-sorted finite ground terms of type T over C. We write $C(\overline{v})$ to denote a value with constructor C and subterms $\overline{v}$ matching the signature of C, and use the notation $|C|$ to mean the arity of C. Since every constructor has finite arity, every value is finitely branching, even though C itself may be countably infinite. We assume two distinguished

types: Bool, with two nullary constructors, true and false, and Int, with one nullary constructor for each integer. We denote the full set of all semantic values as $\mathcal{V}$. We define the function sig : $C \to T^*$ to map constructors to their signatures.

3.2 Sketch-based probabilistic program synthesis

Let $\mathcal{P}$ be a domain-specific probabilistic programming language, and denote $\mathcal{E} \subseteq \mathcal{P}$ as the set of expressions in $\mathcal{P}$. The language $\mathcal{P}$ includes *partial programs*, that is, programs with holes; a *program sketch* in $\mathcal{P}$ may have one or more holes. Programs may also have *symbolic parameters*, which are unknown at compile time. For a program sketch $P \in \mathcal{P}$ with h holes and k symbolic parameters, we write $P[\overline{e}|\theta]$ to mean the complete program obtained by filling the holes in P with a vector of subexpressions $\overline{e} \in \mathcal{E}^h$ and binding the symbolic parameters to the parameter vector $\theta \in \mathbb{R}^k$. Let $[\![\cdot]\!] : \mathcal{P} \to \mathcal{V} \to [0, 1]$ be a semantics for $\mathcal{P}$ that maps each complete program $P^\star$ (without unfilled holes or unbound symbolic parameters) to a sub-probability distribution over semantic values in $\mathcal{V}$. Let $D = \{v_i\}_{i=1}^n$ be a multiset of observations such that each $v_i \in \mathcal{V}$.

The *sketch-based probabilistic program synthesis* problem is to find, given a program sketch p with h holes and k symbolic parameters, a vector of subexpressions $\overline{e}^\star \in \mathcal{E}^h$ and a parameter vector $\theta \in \mathbb{R}^k$ that maximize the likelihood of the observations in D:

$$\langle \overline{e}^\star, \theta \rangle = \arg \max_{\overline{e} \in \mathcal{E}^h, \theta \in \mathbb{R}^k} \prod_{i=1}^{n} [\![P[\overline{e}|\theta]]\!](x_i)$$

Equivalently, the objective can be written as the joint conditional probability of the observed data given a complete parameterized program:

$$\langle \overline{e}^\star, \theta \rangle = \arg \max_{\overline{e} \in \mathcal{E}^h, \theta \in \mathbb{R}^k} \Pr\left(D \mid p[\overline{e}|\theta]\right)$$

4 THE TALI LANGUAGE

In this section we introduce Tali , a probabilistic programming language over inductive datatypes for expressing probabilistic data summaries. We first present the syntax and semantics of our language in Section 4.1 and Section 4.2, respectively. We then describe, in Section 4.3 and Section 4.4, a type system that defines a class of programs amenable to efficient learning: their likelihoods can be computed directly without marginalizing over infinite sets of execution traces.

4.1 Syntax

Figure 2 shows the syntax. Tali is a first-order probabilistic programming language with recursive functions, pattern matching, standard control-flow constructs, variable assignment, and probabilistic choice operators. A program is a sequence of function definitions followed by an expression which represents the main body of the program. Stochasticity is introduced through the probabilistic choice operators bern p and geom p. The bern p expression samples a Bernoulli distribution with parameter p, evaluating to true with probability p and false with probability $1 - p$. Similarly, the geom p expression samples a geometric distribution with parameter p, evaluating to a (zero-indexed) natural number n with probability $p^n(1 - p)$.

Parameters are not first-class values and are fixed at runtime. A given parameter may either be a concrete numerical value θ on the interval $(0, 1)$, or else a symbolic parameter π. Symbolic parameters are mapped to numerical values just like concrete parameters, but their values are supplied by the environment at runtime.

Unlike many other probabilistic languages, we do not support an observe construct. This is primarily because we are specifically interested in learning generator programs from observations.

$$
\begin{array}{rl}
\text{Base Types:} & A ::= \mathsf{Bool} \mid \mathsf{Int} \mid T \\
\text{Modal Types:} & \tau ::= \triangleright^n A \qquad (n \in \mathbb{N},\ \triangleright^0 A \triangleq A) \\
\text{Function Types:} & \sigma ::= \overline{\tau} \to \tau' \\
\text{Parameters:} & p ::= \theta \mid \pi \\
\text{Expressions:} & e ::= x \mid f(\overline{e}) \mid \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \\
& \quad \mid C \mid C(\overline{e}) \mid \mathsf{case}\ e\ \mathsf{of}\ \overline{\{C(\overline{x}) \to e'\}} \\
& \quad \mid \mathsf{bern}\ p \mid \mathsf{geom}\ p \\
\text{Values:} & v ::= C \mid C(\overline{v}) \\
\text{Definitions:} & d ::= \mathsf{define}\ f(\overline{x : \tau}) : \tau' := e \\
\text{Programs:} & P ::= e \mid d;\ P
\end{array}
$$

Fig. 2. Syntax of the core calculus for recursive data sampling.

In this application domain, the main use of observe statements would be to capture logical properties of the data domain (e.g., that the trees are valid binary search trees). Synthesizing logical properties from observations is challenging in its own right, as is designing a language that incorporates both logical and distributional properties (and observe statements may not be the appropriate solution). We leave this problem to future work. Note that one can ensure logical properties in a constructive manner using program patterns that only generate satisfying data values (e.g., always increment to ensure sortedness), and our framework supports this modeling.

Types are stratified into *base* types, which are the ordinary types of values, and *modal* types $\triangleright^n A$, which additionally record that a value is available only n steps "later." We defer full explanation of the later modality to Section 4.4.

4.2 Semantics

A Tali program denotes a sub-probability distribution over values that it can generate. Because programs are recursive and their outputs are inductive, we must account for programs that may diverge or produce infinite values. Consider, for example, the program

$$\mathsf{define}\ \mathsf{gen}() : \mathsf{List} := \mathsf{Cons}(0, \mathsf{gen}());\ \mathsf{gen}()$$

This program clearly does not terminate, and it should assign zero likelihood to every finite list, since all of its probability mass rests on infinite lists. Because our semantic domain is restricted to the finite values in the term algebra induced by the set of constructors, the naïve semantics over complete values are ill-suited to handle all Tali programs. To formalize the semantics of Tali , we instead interpret a program as a distribution over *partially evaluated values*, whose as-yet-unproduced parts are represented by a placeholder $\bot$. The (sub-)probability distribution over complete values (and thus over observed values) is then the special case over $\bot$-free values.

Definition 4.1 (Partial value)**.** A *partial value* is a value in which some subterms may be replaced with $\bot$, meaning "not yet evaluated."

$$\delta ::= \bot \mid C(\overline{\delta})$$

We define an ordering on partial values $\delta_1 \sqsubseteq \delta_2$, read "$\delta_1$ is no more defined than δ_2," such that $\bot \sqsubseteq \delta$ for all δ, and $\delta_1 \sqsubseteq \delta_2$ if $\delta_1 = C(\overline{\delta_{1i}})$ and $\delta_2 = C(\overline{\delta_{2i}})$ where $\delta_{1i} \sqsubseteq \delta_{2i}$ pointwise. Values with

$$\mu[\![C(\overline{e})]\!]_{\rho,\phi}(C(\overline{\delta})) = \prod_{i=1}^{|C|} \mu[\![e_i]\!]_{\rho,\phi}(\delta_i) \qquad \mu[\![C(\overline{e})]\!]_{\rho,\phi}(C'(\overline{\delta})) = 0,\ C \neq C' \qquad \mu[\![C(\overline{e})]\!]_{\rho,\phi}(\bot) = 0$$

$$\mu[\![x]\!]_{\rho,\phi}(\delta) = \begin{cases} 1 & \rho(x) = \delta \\ 0 & \text{otherwise} \end{cases} \qquad \mu[\![\text{bern } p]\!]_{\rho,\phi}(\delta) = \begin{cases} p & \delta = \text{true} \\ 1-p & \delta = \text{false} \\ 0 & \text{otherwise} \end{cases}$$

$$\mu[\![\text{geom } p]\!]_{\rho,\phi}(\delta) = \begin{cases} (1-p)p^{\mathcal{N}[\![\delta]\!]} & \delta = 0, 1, 2, \ldots \\ 0 & \text{otherwise} \end{cases}$$

$$\mu[\![f(\overline{e})]\!]_{\rho,\phi}(\delta) = \sum_{\overline{\delta'} \in \mathcal{V}_\bot^{|f|}} \left(\prod_{i=1}^{|f|} \mu[\![e_i]\!]_{\rho,\phi}(\delta_i') \right) \cdot \phi(f)(\overline{\delta'})(\delta)$$

$$\mu[\![\text{let } x = e_1 \text{ in } e_2]\!]_{\rho,\phi}(\delta) = \sum_{\delta' \in \mathcal{V}_\bot} \mu[\![e_1]\!]_{\rho,\phi}(\delta') \cdot \mu[\![e_2]\!]_{\rho[x \mapsto \delta'],\phi}(\delta)$$

$$\mu[\![\text{case } e \text{ of } \overline{\{C_i(\overline{x_i}) \to e_i'\}}^n]\!]_{\rho,\phi}(\delta) =$$

$$\sum_{i=1}^{n} \sum_{\overline{\delta'} \in \mathcal{V}_\bot^{|C_i|}} \mu[\![e]\!]_{\rho,\phi}(C_i(\overline{\delta'})) \cdot \mu[\![e_i']\!]_{\rho[\overline{x} \mapsto \overline{\delta}],\phi}(\delta) + \begin{cases} \mu[\![e]\!]_{\rho,\phi}(\bot) & \delta = \bot \\ 0 & \text{otherwise} \end{cases}$$

Fig. 3. Denotational semantics of the Tali language. Each expression denotes a measure $\mu_{\rho,\phi}[\![e]\!]$ on partial values $\delta \in \mathcal{V}_\bot$. $\mathcal{N}[\![\delta]\!]$ is a partial function which maps syntactic integers to $\mathbb{Z}$. We write $|f|$ to mean the arity of a function f (that is, $|f| = |\overline{\tau}|$ if $\Sigma(f) = \overline{\tau} \to \tau'$).

distinct constructors are incomparable. We say δ_1 and δ_2 are *compatible* if they share an upper bound; intuitively, this means that they agree on their top-level constructors and all transitive subterms, up to $\bot$. We define $\mathcal{V}_\bot$ to mean the set of all partial values generated by the fixed signature as described in Section 3. The members of $\mathcal{V}_\bot$ are finite, so $\mathcal{V}_\bot$, like $\mathcal{V}$, is countable. The set of semantic values $\mathcal{V} \subseteq \mathcal{V}_\bot$ is then the special case of $\bot$-free (and thus maximal) partial values. We refer to such values $v \in \mathcal{V}$ as *complete values*.

4.2.1 Probability distribution over partial values. A program $P = \overline{d}; e$ denotes a discrete sub-probability distribution over $\mathcal{V}_\bot$. Its denotation gives the likelihood that execution of P eventually produces a partial observation δ. To formally construct this denotation, we first define a measure on $\mathcal{V}_\bot$ denoted by an expression e. The equations are given in Figure 3. The measure $\mu[\![e]\!]_{\rho,\phi}$ is indexed by a variable context $\rho : \text{Name} \rightharpoonup \mathcal{V}_\bot$ and a function context $\phi : \text{Name} \rightharpoonup (\mathcal{V}_\bot^* \to \mathcal{V}_\bot \to [0,1])$. Note that the variable context ρ binds variables to partial values, not to distributions over partial values. Threading dependent state is handled through marginalization in the equations for the binding forms. This distinction matters because it prevents incorrect denotations of expressions such as let x = bern p in Pair(x, x). Were a naïve distribution-driven approach used, we would need to take care to ensure that the pair's members are correctly correlated and not independent Bernoulli trials. The state-threading approach naturally sidesteps such pitfalls.

For a well-formed expression e in which every case subexpression is exhaustive, the measure in Figure 3 is a probability distribution:

Lemma 4.2 (Probability distribution). *Let e be a well-formed expression. Let ρ be a variable context defined on the free variables of e, and let ϕ be a function context whose values are probability distributions. Then $\mu[\![e]\!]_{\rho,\phi}$ is a probability distribution over $\mathcal{V}_\perp$; that is,*

$$\sum_{\delta \in \mathcal{V}_\perp} \mu[\![e]\!]_{\rho,\phi}(\delta) = 1.$$

We can now define the likelihood that a program is *compatible* with a particular finite, possibly partial observation. For a partial value δ, we interpret its upward closure $\uparrow \delta = \{\delta' \in \mathcal{V}_\perp \mid \delta \sqsubseteq \delta'\}$ as the set of partial values compatible with the prefix δ; we call this set a *demand*. The denotation of an expression e on a partial value δ is the measure of $\uparrow \delta$. We write $[\![e]\!]_{\rho,\phi}(\delta)$ for the likelihood that e meets the demand δ:

$$[\![e]\!]_{\rho,\phi}(\delta) = \mu[\![e]\!]_{\rho,\phi}(\uparrow \delta) = \sum_{\delta_i \in \uparrow \delta} \mu[\![e]\!]_{\rho,\phi}(\delta_i)$$

Since $\uparrow \perp = \mathcal{V}_\perp$ and $\mu[\![e]\!]_{\rho,\phi}$ is a probability distribution on $\mathcal{V}_\perp$, we have $[\![e]\!]_{\rho,\phi}(\perp) = 1$. Furthermore, because every value in $\mathcal{V}$ is maximal, the two denotations agree exactly on complete values: indeed, if $\delta \in \mathcal{V}$, then $\uparrow \delta = \{\delta\}$, and so $\mu[\![e]\!]_{\rho,\phi}(\delta) = [\![e]\!]_{\rho,\phi}(\delta)$. Since $\mu[\![\cdot]\!]_{\rho,\phi}$ is a probability distribution on $\mathcal{V}_\perp$, $[\![\cdot]\!]_{\rho,\phi}$ is a sub-probability distribution on $\mathcal{V}$. We will focus on $[\![\cdot]\!]_{\rho,\phi}$ for the rest of this paper, as it is the value that we compute to evaluate the likelihood of a fully observed set of values for a given program.

4.2.2 Function fixed points and approximants. Next, we define the semantics of recursive functions. We handle recursion in the standard way, by taking a fixed point of a function context. For a program P, written

$$P = \text{define } f_1(\overline{x_1 : \tau_1}) : \tau_1' \coloneqq e_1;\ \ldots\ \text{define } f_m(\overline{x_m : \tau_m}) : \tau_m' \coloneqq e_m;\ e_{\text{main}},$$

we define a functional whose value for each f_i, with parameters $x_{i1}, \ldots, x_{ik_i}$ and $\overline{\delta} \in \mathcal{V}_\perp^{k_i}$, is:

$$\Phi(\phi)(f_i)(\overline{\delta})(\delta') = \mu_{\rho_i,\phi}[\![e_i]\!](\delta') \quad \text{where } \rho_i = \{x_{ij} \mapsto \delta_j\}_{j=1}^{k_i}$$

We define $\phi_0(f)(\overline{\delta})$ as the point mass on $\perp$:

$$\phi_0(f)(\overline{\delta})(\delta) = \begin{cases} 1 & \delta = \perp \\ 0 & \text{otherwise} \end{cases}$$

and define $\phi_{k+1} = \Phi(\phi_k)$. Operationally, the context ϕ_k represents iteratively "unrolling" each function call k times, placing any residual mass on $\perp$ at the base case. Each unrolled context is an approximant of the fixed point of Φ; we refer to each ϕ_k as the k-th approximant of the function environment.

The approximants need not converge pointwise as distributions, but their likelihoods converge monotonically.

Theorem 4.3 (Refinement). *Let $k \le k'$ and $\rho \sqsubseteq \rho'$ pointwise. Then for any expression e, there exists a coupling of $\mu_{\rho,\phi_k}[\![e]\!]$ and $\mu_{\rho',\phi_{k'}}[\![e]\!]$, supported on pairs (δ, δ'), such that $\delta \sqsubseteq \delta'$. Thus $[\![e]\!]_{\rho,\phi_k}(\delta) \le [\![e]\!]_{\rho',\phi_{k'}}(\delta)$ for each demand δ, and approximants $[\![e]\!]_{\rho,\phi_k}$ are non-decreasing in k.*

A consequence of Theorem 4.3 is that repeatedly unrolling function applications only refines the output; it never revises what is already defined. Thus we may define

$$[\![e]\!]_\rho(\delta) = \lim_{k \to \infty} [\![e]\!]_{\rho,\phi_k}(\delta) = \sup_k [\![e]\!]_{\rho,\phi_k}(\delta)$$

as simply the limit of a bounded monotone sequence of reals. The semantics of a full program P is then

$$[\![P]\!](\delta) = [\![e_{\text{main}}]\!]_{\varnothing}(\delta),$$

and we write the k-th approximant of the semantics (i.e. the semantics evaluated under function environment ϕ_k) as

$$[\![P]\!]_k(\delta) = [\![e_{\text{main}}]\!]_{\varnothing,\phi_k}(\delta).$$

The semantics of Tali are well-defined for all recursive programs and partial values, although not all programs generate complete, $\bot$-free values $v \in \mathcal{V}$ with probability 1. Probability mass can escape from $\mathcal{V}$ in two ways. First, a program may encode a supercritical branching process which diverges with positive probability, or equivalently places mass on infinite trees (which we represent as sets of ever-deeper partial values truncated at $\bot$ leaves). Such programs may still produce finite values, to which our semantics correctly assigns likelihoods; these likelihoods simply do not sum to 1. Second, a program may produce no output, looping forever before deciding its first constructor. Such programs place some or all of their probability mass on $\bot$; they are pathological because computing a likelihood requires marginalizing over a potentially unbounded set of execution traces consistent with the observation. In the following section, we define a semantic program property, *observation-bounded compilability*, that rules out such programs, along with a type system that accepts only well-behaved programs.

4.3 Observation-Bounded Compilability

Our language features unbounded recursion in order to concisely express recursive generative processes over inductive values. However, unbounded recursion can be problematic and difficult to deal with. To guarantee that bounded unrolling suffices to compute a recursive program's distribution, we must ensure that unobserved probabilistic choices are well-behaved and can be marginalized exactly.

It is therefore desirable to identify a class of probabilistic programs with this property. One common approach to recursion in probabilistic languages is a fixed unroll or iteration bound. While this approach can help simplify implementation, it can produce incorrect likelihoods if the program's approximated likelihood does not stabilize at some finite unroll bound. Worse, the minimum unroll bound needed to compute a value's likelihood exactly may be difficult to derive. For instance, with fixed unrolling, one could increase the bound until observation likelihoods stabilize. However, this could require recomputing the same likelihoods many times and might not converge for all recursive programs. (As an example, programs with non-terminating recursion will never see their observed likelihoods plateau at the exact probability mass, though they may approach it asymptotically.) Thus, it is desirable to statically ensure that *all* Tali programs admit exact inference using a simple unrolling scheme.

Formally, we define the following *observation-bounded compilability property* for recursive programs. It characterizes programs for which inference can be performed exactly by unrolling function calls up to a bound determined by the observation.

Definition 4.4 (Observation-bounded compilability)**.** A program P is *observation-bounded compilable* if and only if there exists a total computable function $\beta_P : \mathcal{V} \to \mathbb{N}$ such that

$$\forall v \in \mathcal{V}, k \geq \beta_P(v) : [\![P]\!]_k(v) = [\![P]\!](v).$$

Intuitively, the sorts of programs which satisfy this property are programs for which each recursive call is accounted for in the output. Each "layer" of an observed value can be thought of as the result of a bounded set of probabilistic choices, so the set of program traces consistent with an observation can be recovered by iteratively "peeling off" constructors and recording which

choices were made at each step. This structure aligns naturally with real-world recursive samplers. It rules out certain program behaviors, such as infinite loops or filters over infinite streams or state spaces, that could harm performance. By imposing this restriction, we apply an inductive bias which guides synthesis towards programs that provide algorithmic or constructive explanations in terms of generative processes, rather than declarative logical invariants.

One important note is that the observation-bounded compilability property does not make probabilistic inference tractable in general. Discrete probabilistic inference over boolean domains is #P-hard [48], and OBC does not ameliorate this. The OBC property merely provides a bound on the length of an execution trace for an arbitrary finite observation.

4.4 Guarded recursion

Enforcing OBC requires a single semantic restriction: every recursive call must produce observable output. Since the size of an observation is finite, it follows that its size bounds the number of recursive calls needed to produce it. Fortunately, guarded recursion provides a simple way to enforce this semantic property: every recursive call must be enclosed by one or more constructor applications.

To see why guardedness is needed, consider the following function definition, which recursively constructs a list of boolean values.

```
define buildList () : BoolList =
  if bern p then
      Cons(bern q, buildList()) else if bern r then
      Nil else
      buildList()      ← unguarded recursion
```

The final recursive call to buildList is *unguarded* because it is returned directly, without a constructor. Suppose we wish to compute the likelihood of a given list, say Cons(true, Nil). There are many unrollings consistent with the observation; for instance, it is possible that bern p, bern q, and bern r all produced true, resulting in the observation. However, bern r could also produce false any number of times before producing true. Thus, infinitely many traces are consistent with the observation, and no finite unrolling can compute its likelihood exactly.

We formalize this principle with a modal type system; selected rules are shown in Figure 4. Since functions in Tali are first-order, programs are typed against a function signature Σ, which is a finite map from (unique) function names to type signatures. Functions are added to Σ in order of definition; mutual recursion is not directly supported.[1] We use the top-level judgment $\Sigma \vdash P : \tau$ to state a program's type: under function signature Σ, program P is well-typed and produces a value of type τ. Expressions are typed with the judgment

$$\Sigma; \Gamma; f \vdash e : \tau,$$

meaning that under function signature Σ, variable context Γ, and enclosing function name f, the expression e has type τ. We write $f = \cdot$ when e is the main body of a program rather than a function body.

We now implement guarded recursion using the modal features of Tali 's type system.

4.4.1 Later modality. To enforce guardedness, Tali 's type system includes a clocked "later" modality, written $\triangleright^n A$. Our approach is closely related to those of Nakano [40] and Atkey and McBride [3], where similar systems are used to prove the productivity of corecursive computations. Intuitively,

[1] Note that mutual recursion can be emulated using single recursion and a tag argument.

a value at later index n is one that will only be available after n further steps; a value at index 0 is available "now." Types are stratified; that is, $\triangleright^n$ is a prefix attached to a base type rather than a type former in itself. Consequently, there are no nested modalities (e.g. $\triangleright^m(\triangleright^n A)$), though modalities may be shifted (e.g. $\triangleright^{m+n} A$). Furthermore, since we take $\triangleright^0 A$ to be literally the base type A, the types A and $\triangleright^0 A$ may be exchanged freely.

The guardedness property now follows from a short argument. The index is raised by T-Call in recursive contexts, by T-Case in case destructuring, and by T-Later-Sub as a subsumption mechanism; it is lowered only by T-Constr. Since the modal index of the return value of a recursive call is raised by at least 1, an unguarded recursive call cannot be returned directly since a modal type is never subsumed by a lower modal index, only by a higher one. Furthermore, a recursive self-call cannot be "laundered" by wrapping it in a constructor and immediately destructuring it with a case expression; doing so leaves the modal index at its incremented value. Thus every self-call occurring in a well-typed body must be enclosed in at least one constructor application before being returned, which is precisely what the buildList example above violates. Its final buildList() call is returned directly without being wrapped in a Cons, and this fails to type-check since the call is typed at index 1 in a position demanding index 0.

4.4.2 Typing judgments. The selected rules of Figure 4 show how the modal index is manipulated. T-Later-Sub defines the subsumption discipline: types may always be used at a later index (though not at an earlier one). The modal index is incremented by function calls and case destructuring. T-Call types recursive calls at a higher modal index, thereby incurring some amount of "modal debt." (Non-recursive calls are typed at the same index, merely passing the index of their arguments through to its result.) T-Case binds the children of a scrutinee at modal index n to variables at modal index $n+1$. This is necessary since the scrutinee's children may not be available yet, despite the scrutinee's root constructor being available; in effect, performing case analysis defers the modal obligation by one step. T-Constr is the sole elimination rule for modal types: a constructor applied to arguments at index $n+1$ yields a result at index n, discharging one unit of modal debt in exchange for one layer of observable output.

4.4.3 Type soundness. We now state the main soundness property of the type system: well-typed programs exhibit the OBC property.

Theorem 4.5 (Soundness)**.** *For any program P and type τ, if $\cdot \vdash P : \tau$, then P is OBC.*

A key consequence of OBC is that probabilistic inference for such programs can be implemented by unrolling function definitions to a depth determined by the observation. In the following section, we briefly describe a straightforward inference procedure for well-typed programs.

4.5 Probabilistic inference for Tali Programs

Since OBC Tali programs are guarded and productive by construction, every well-typed program reduces to weak-head normal form (WHNF) in finite time. In our denotational semantics, this means that every well-typed program assigns zero probability to $\bot$, even if it does not terminate completely. As a result, the likelihood of a given value can be computed as the sum of likelihoods over a finite set of demands. Operationally, we implement probabilistic inference for Tali programs using lazy enumeration [31, 32]. We construct thunks for each probabilistic draw and recursive function call, and force only when needed, such as in control flow. Inference proceeds "outside-in" on a given value: we force the root constructor before its children and continue recursively in a top-down fashion. Thunking probabilistic draws also allows for certain language extensions to be implemented easily, such as offsetting and clamping integer draws.

$$\textsc{T-Later-Sub}\quad \frac{\Sigma;\Gamma;f \vdash e : \triangleright^m A \qquad m \leq n}{\Sigma;\Gamma;f \vdash e : \triangleright^n A} \qquad \textsc{T-Constr}\quad \frac{C \in C_A \qquad \mathrm{sig}(C) = (A_1,\ldots,A_k) \qquad \forall i \leq k.\ \Sigma;\Gamma;f \vdash e_i : \triangleright^{n+1} A_i}{\Sigma;\Gamma;f \vdash C(\overline{e}) : \triangleright^n A}$$

$$\textsc{T-Call}\quad \frac{\begin{array}{c} j \geq 1 \text{ if } g = f \text{ else } j \geq 0 \\ \Sigma(g) = (\triangleright^{m_1} A_1,\ldots,\triangleright^{m_k} A_k) \to \triangleright^n A' \qquad \forall i \leq k.\ \Sigma;\Gamma;f \vdash e_i : \triangleright^{m_i+j} A_i \end{array}}{\Sigma;\Gamma;f \vdash g(\overline{e}) : \triangleright^{n+j} A'}$$

$$\textsc{T-Case}\quad \frac{\begin{array}{c} \Sigma;\Gamma;f \vdash e : \triangleright^n A \qquad C_A = \{C_1,\ldots,C_k\} \qquad \forall i \leq k.\ \mathrm{sig}(C_i) = (A_{i1},\ldots,A_{ik_i}) \\ n \leq m \qquad \forall i \leq k.\ \Sigma;\Gamma,x_{i1} : \triangleright^{n+1} A_{i1},\ldots,x_{ik_i} : \triangleright^{n+1} A_{ik_i};f \vdash e_i : \triangleright^m A' \end{array}}{\Sigma;\Gamma;f \vdash \text{case } e \text{ of } \overline{\{C_i(\overline{x_i}) \to e_i\}}^{k} : \triangleright^m A'}$$

Fig. 4. Selected typing rules for the Tali language. A complete listing of the typing rules can be found in Appendix C.

The inference procedure does not simply compute a numeric solution to the semantic equations in Figure 3. Instead, each forcing step produces one or more nodes in an arithmetic circuit that represents the semantic equations symbolically in terms of the probabilistic parameters. This allows for efficient parameter tuning, which is exploited during synthesis (see Section 5.1.4). The OBC guarantees are sufficient for this to be feasible. The arithmetic circuit for the probability mass that a program assigns to an observation can be decomposed into a sequence of recursive-call subcircuits. Since the number of recursive calls is bounded by the size of the observation, the number of sub-circuits is similarly bounded, and since the set of demands at each recursive level is finite, each sub-circuit also has finite size.

5 LEARNING STOCHASTIC SUMMARIES

To automatically construct generative models of data, we use program synthesis to search for program structures with high marginal likelihood. After identifying a likely model, we optimize its parameters to fit the data using maximum likelihood estimation (MLE).

5.1 Synthesis of probabilistic programs

To synthesize a probabilistic program that best describes a given dataset, we use a sketch-based stochastic synthesis algorithm. Stochastic search and Monte Carlo techniques are well established tools for probabilistic program synthesis [42, 49]. The inputs to the synthesis algorithm are a user-provided program sketch $\mathcal{P}$ with one or more holes, and a training set of observations. Given these inputs, the synthesis algorithm searches for a complete program $\mathcal{P}[\overline{e}|\theta]$, with filled-in expressions $\overline{e}$ and parameter vector θ that maximize the likelihood of the observations.

5.1.1 Likelihood evaluation and program scoring. Program synthesis is driven by a likelihood-directed program scoring function. Programs that explain the data well have high marginal likelihood, defined as the integral of the program's observation-conditional likelihood over the parameter space. The Tali evaluator computes an arithmetic circuit for the likelihood of a set of observations under a given parameter assignment. This makes marginal-likelihood computation fairly straightforward, though not always tractable. In the special case where the likelihood circuit is a monomial, every observation is unambiguous—that is, a unique set of random choices during

execution could have produced it. For such circuits, the exact marginal likelihood can be computed in closed form with beta priors over each Bernoulli sample in the trace. For general polynomial circuits, the marginal likelihood may be intractable, so outside of the monomial case we use mean-field variational Bayes to compute an evidence lower bound (ELBO) for the candidate program [5, 7]. One consequence of this approach is that the marginal-likelihood score favors unambiguous programs with monomial likelihoods.

Each program's score combines its marginal-likelihood score with the syntax-tree size of its hole fillings. We also add a termination penalty for programs that are not almost-surely terminating. Well-typed Tali programs are productive by construction, but this does not imply almost-sure termination; for example, a candidate programs may implement a supercritical branching process which diverges with nonzero probability. Nonterminating programs are undesirable as generative models. However, because our lazy-enumeration inference technique prunes paths based on the observations, it cannot distinguish a program that diverges on part of its support from one that terminates but fits the data poorly on that same part. Thus we penalize each proposed candidate proportional to its divergence probability to down-weight non-terminating programs. The divergence probability of an arbitrary program is uncomputable in general, so we approximate it empirically by drawing samples from the program using a fixed execution budget, and calculate the proportion of executions that exhaust the budget.

The scoring function S for a given program P with hole fillings $\overline{e}$ and parameter assignment θ is then

$$S(P[\overline{e}|\theta]) = -\log \widehat{p}(\mathcal{D} \mid P) + \log |\overline{e}| + T(P),$$

where $|\overline{e}|$ is the total syntax-tree size of the hole fillings, T is the empirical termination penalty, and $\log \widehat{p}$ denotes exact log evidence in the monomial case and its variational lower bound otherwise.

5.1.2 Initial program search. The synthesis procedure consists of an enumerative search over a feasible set of recursive program structures, followed by a Metropolis-style stochastic search guided by the likelihood of the observations under each model. In our evaluations, we synthesize programs from sketches which contain at least one recursive function, with each entire function body left as a hole to be filled.

Initialization searches a restricted family of recursive programs for a promising starting program with nonzero likelihood on the training observations. This is essential since the joint likelihood gives no partial credit for explaining only part of the dataset: omitting even one observed constructor assigns the dataset probability zero. Most randomly generated bodies therefore receive the same infinite score, leaving pure stochastic search with no useful signal from the scoring function. We instead search for an initial *covering skeleton*: an initial hole filling which assigns non-zero probability to every observation.

For a given output type, a basic skeleton contains a cascade of probabilistic choices with one alternative per observed constructor. Inductive constructor fields are filled by recursive calls, integer and boolean fields by primitive samples, and flat fields by a choice over their enumerated constructors. For instance, consider a recursive Boolean list generator over constructors Nil and Cons. Its basic skeleton consists of a single-choice cascade covering Nil and Cons, where the Cons fields are filled with a primitive bern and a recursive call for its tail.

A sketch typically has one generator per type, so every call to it draws from the same constructor distribution, regardless of where in the tree the call occurs. This cannot express many context-sensitive structures, such as a tree where children can be drawn from different distributions depending on the parent's label. Before building the basic skeletons, we optionally split each generator by call-site, following parent annotation [29]. A call site is either the root of an observation or a constructor field (for example, the left child of a Node). Because each subterm's call site is visible

in the data, we can quickly compute the skeleton's marginal likelihood in closed form from per-site constructor counts and field values under Beta priors, without running the full evaluator. We begin with one class per site, and, following Bayesian model merging [28, 53], greedily merge classes by marginal likelihood, stopping when no candidate merger would increase it. Each remaining class becomes a constructor of a new tag type, which is passed to the skeleton as an additional argument. The function takes this tag as an additional argument and matches on it, producing an independent sub-skeleton cascade for each tag constructor. Each cascade arm passes the tag as an argument to its recursive calls.

Within each skeleton, we choose an expression for each integer field and the arguments for each recursive call. The grammar includes multiple forms for integer expressions, such as geometric draws (optionally offset or clamped) and mixtures over point masses and geometric draws; the full skeleton grammar is given in Section D.1. Generated fields are bound in let expressions so that their values may be shared in subsequent expressions. Each field therefore has a finite set of choices that can be enumerated during search.

These choices must often be made jointly. To illustrate, consider a sketch for a sorted-list generator $\mathrm{sorted}(l : \mathsf{Int}, h : \mathsf{Int})$, where the arguments are the current lower and upper bounds. Generating the head $x = \min(\mathrm{geom}\ \theta + l, h)$ and passing x as the lower bound in the recursive call $\mathrm{sorted}(x, h)$ produces an ordered list within the interval. These choices must be made together, and the variable x must be threaded correctly. Searching these choices jointly exposes useful dependencies that isolated subtree changes might not discover.

The product of this choice space is finite but can be large. We fix an evaluation budget; if it exceeds the size of the choice space, we enumerate the space exhaustively. Otherwise, we enumerate field forms and perform coordinate descent over call arguments for each combination. If this is still too expensive, we perform coordinate descent over all choices, starting from the default seed with bare draws and default argument passing. Each sweep retains strictly improving changes and stops when a fixed point is reached. The best-scoring seed initializes the chain, and we retain the evaluated assignments for restarts (see below). This search offers no guarantee of global optimality when enumeration is incomplete.

5.1.3 Stochastic search. Starting from the selected seed, stochastic search explores the program space using a subtree replacement kernel. Each generated subtree records its target type, in-scope variables, guardedness, and outstanding relevance obligations. A replacement is generated in that saved context with fresh local binders, preserving the expression's type and variable bindings. Per the type system, recursive self-calls are permitted only in constructor-guarded positions. Subtrees must also satisfy a variable-use requirement, following relevant typing [47]: each inductive function argument must be used at least once in every control-flow branch. The generator chooses among admissible typing rules and fills their subexpressions recursively, backtracking when a rule's obligations cannot be satisfied. Consequently, the search explores a restricted subset of well-typed Tali programs which expresses a variety of recursive structures.

The stochastic search mixes structural and local proposals. At each step, the search selects either structural sites (conditionals, matches, constructor applications, and let bindings) or argument sites (calls, primitive operations, and atomic expressions). Sites are selected uniformly within the chosen class; if that class is empty, the search may use any non-root replacement site. If local replacement fails, the selected hole is regenerated wholesale. The probability of choosing a structural site rather than an argument site decays during the search. Thus, early proposals more often revise branching and sharing structure, while later proposals emphasize data flow without eliminating structural exploration.

To avoid getting stuck in local minima, the search can periodically regenerate a hole's entire filling. Fresh restarts are drawn without replacement from the searched seed assignments, weighted by their scores. Alternatively, the search can make a fresh type-directed draw for the hole.

Each proposed program candidate is scored with the scoring function S described in Section 5.1.1. For finite scores, a proposed program P' replaces the current accepted program P with probability

$$A(P, P') = \min\{1, \exp(S(P) - S(P'))\}.$$

Proposals with infinite score are rejected from a finite-score state; when both scores are infinite, the proposal is accepted to permit continued exploration. Because the proposals are asymmetric and change with the search schedule, and because the acceptance rule omits a proposal-ratio correction, this is *not* a posterior sampler over the program space. Rather, it is a Metropolis-style stochastic search for program selection.

5.1.4 Parameter fitting. After stochastic search is complete, we fit parameters for the top-k program structures found during the search. Since the inference algorithm constructs an arithmetic circuit representing the likelihood of the observations given the program parameters, we are able to compute gradients of the likelihood function and optimize the parameters using L-BFGS, a gradient-based optimization algorithm [36]. After optimization, each of the top-k programs is scored against the validation set, and the best program is returned.

6 EVALUATION

We evaluate Tali on a synthetic benchmark grid with data generated from known ground truth programs and on two held-out real-world domains, regular expressions [13] and SPARQL queries [37]. We hypothesize that, with tens to hundreds of training trees, the structural prior of the OBC program class can allow synthesized programs recover the structure of a distribution from far fewer examples than data-intensive models such as transformers.

We examine the following research questions in our evaluation:
RQ1: How well do synthesized programs fit the data distribution, compared with matched-data neural baselines, and how does this change with training-set size in the low-data regime?
RQ2: Which invariants of the data do synthesized programs recover, and how much training data is needed for reliable learning?
RQ3: Can synthesized programs act as a data augmentation source to improve the performance of downstream model training tasks on real-world data?

6.1 Benchmarks

6.1.1 Synthetic grid. The grid (Table 1) comprises 21 ground-truth program grammars over three data domains: strings, binary trees, and expression ASTs. The *structure* axis (S1–S4) varies how the ground truth recurses; the *value-relation* axis (V1–V3) varies how far a dependency between values reaches: to a neighbor, to every ancestor, or to an aggregate of a whole subtree. S1–S4 and V2 are naturally expressible as OBC programs; V1 requires threading a value through the recursion, and V3 depends on subtrees not yet generated. Known ground truths let us measure fit as excess NLL over the truth, and validity as membership in the truth's support.

6.1.2 Real-world data. The regex dataset is derived from a corpus collected in the empirical software engineering literature [13]; the SPARQL dataset is derived from a collection of queries to Wikidata [37]. We deduplicated, normalized and converted both to ASTs, yielding over 414,000 and 148,000 unique syntax trees, respectively. Each regex character is encoded by its class and its index within that class, so a program must learn the exact distribution over the characters of fixed

	String (D1)	Tree (D2)	Expr. AST (D3)
Geometric (S1)	Stop, or emit a symbol and recurse.	Labelled leaf, or branching node.	Literal, or recursive operator.
Fuel (S2)	Fuel bounds length; early stop allowed.	Fuel bounds depth; children get less.	Fuel bounds depth across both sorts.
State (S3)	Latent state drives stopping and symbols.	Latent state drives stopping and leaves; transitions label children.	Latent state drives literals and operators; transitions follow child position.
Constraint (S4)	Inherited interval; each symbol raises the floor.	Inherited interval; a pivot splits it between children.	Inherited scope removes operators; some tighten it.
Local (V1)	Each symbol within c of the previous.	Each key within c of its parent's.	Literals within c of the enclosing operator's value.
Interval (V2)	Sorted list (as S4).	Search tree (as S4).	Variable indices name an enclosing binder.
Aggregate (V3)	Head holds the list's length.	Each node holds its subtree's size.	Each operator holds the sum of its operands.

Table 1. Synthetic benchmark grid. Rows vary recursive structure (S) and value relation (V), columns vary data domain.

strings. Regex results use uniform samples of the corpus, except for rules about rare constructs (bounded repeats, set ranges, backreferences), which we measure on training sets drawn from regexes containing those constructs.

6.2 Experimental Setup and Baselines

Our method fits a program to each set of n training trees using the synthesis algorithm of Section 5. We then pretrain a transformer on 10,000 samples from the synthesized program and fine-tuned on the original n trees (*program-pretrained*). We compare against two transformers trained on the same n trees: one from scratch (*no pretraining*), and one first pretrained on 10,000 random well-typed values (*random-pretrained*). As references, we use the ground truth on the grid and, on real data, a transformer trained on the entire training pool (*full pool*), which sees orders of magnitude more data than any other approach. We use $n \in \{10, 30, 100, 300\}$ and 30 seeds per setting (27 for regex), and report medians across seeds, means for rates (validity, violation and recall), and *win fractions*: the fraction of seeds on which one arm beats another on the same training set.

6.2.1 Fit metrics. A synthesized program can assign a test tree probability zero, so for a program p and comparison model r we report the paired excess NLL over the test trees X_p that p covers:

$$\frac{1}{|X_p|} \sum_{x \in X_p} \big(-\log[\![p]\!](x) + \log r(x)\big), \qquad X_p = \{x \in X : [\![p]\!](x) > 0\}$$

Alone, this can overstate the fit of a program that covers few trees; as an extreme example, consider a program which assigns very high likelihood to exactly one test tree, but zero likelihood to the rest. Such a program would have perfect *covered* excess NLL, despite severely overfitting. To mitigate this, we always report excess NLL beside *support recall* and, for structure, *support precision*, where q is the data distribution:

$$\mathrm{recall}(p, q) = \Pr_{x \sim q}\,(p(x) > 0) \qquad \mathrm{prec}(p, q) = \Pr_{x \sim p}\,(q(x) > 0)$$

We estimate recall as the fraction of test trees with finite NLL; transformers have recall 1 by construction.

6.2.2 Structure metrics. Invariants are a matter of precision: a model that "knows" lists are sorted never emits an unsorted list. On the grid, we score 1,000 samples per model under the ground

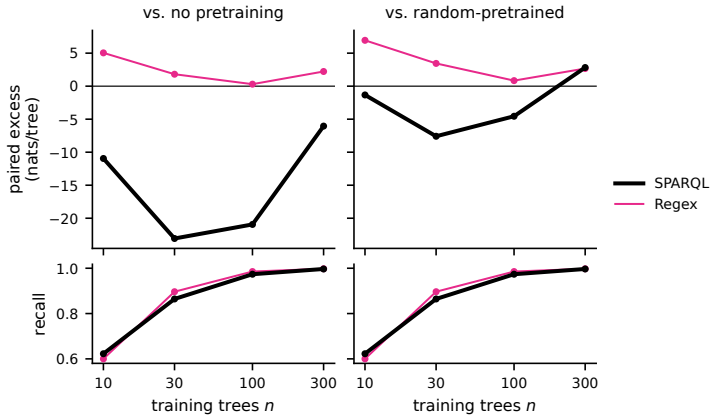


Fig. 5. Median paired excess NLL of synthesized programs versus matched-n transformers, and mean program support recall on the test set (across seeds). Left: Paired excess NLL and recall of the synthesized program versus the no-pretraining model. Right: Paired excess NLL and recall of the synthesized program versus the random-pretraining model.

truth. Every model, the truth included, often replays training trees (55–79% of the truth's samples), which are valid by construction, so we report violation rates on *novel* samples only, and count transformers' malformed outputs (token sequences that do not decode to a tree) separately. For real data, we check domain validity rules instead: for regexes, compilation by Python's `re` module, $m \leq n$ in bounded repeats `{m,n}`, ordered set-range bounds, and backreferences to closed groups; for SPARQL, well-formedness and four scoping rules from the specification (Figure 6).

6.2.3 Held-out data. The grid was our development set. The real-world experiments were preregistered after the method was frozen; we added the program-pretraining arm and the regex matched-n baselines before examining any results. Three regex seeds informed synthesizer design and are excluded from all reported results. SPARQL was never examined during development.

We evaluate RQ1 and RQ2 separately for the the synthetic dataset and real-world datasets in Sections 6.3 and 6.4. We then dedicate Section 6.5 to addressing RQ3.

6.3 Learning Programs over the Synthetic Grid

The results of the evaluation on the synthetic grid are in Table 2. Each cell in the grid corresponds to a set of ground truth programs, some of which are easily expressed in the synthesis DSL and others which are less so. In cases where the synthesis DSL can directly express the ground truth program, programs are more sample-efficient models than either transformer baseline at the same training set sizes . As a particularly stark example, the program's median excess over the truth on cells S1–S3 falls to nearly zero at only $n = 100$: it effectively recovers the exact generating distributions, which are variants on straightforward and productive "sample-then-recurse" patterns. On many grid cells with additional constraints, such as S4 and V2 which enforce value orderings, its excess is only a third to two fifths of the random-pretrained transformer's at $n = 300$, and it fits better on 74–93% of runs at $n \geq 100$.

Probabilistic programs also typically achieve high recall across all grid cells. At $n = 300$, programs reach median recall greater than 99% on 10 of the 21 cells. This is partly an effect of the skeleton

seeding process; a skeleton has full recall by construction, and stochastic search generally stays within a high-recall region of candidate programs.

In the very-low data regime, program synthesis is able to cover a large portion of the support, though its fit is not much better than the baselines. At $n = 10$, the program misses 7–12% of test trees and is only level with random pretraining on the rest.

Constraint recovery on the synthetic grid, much like model fit, is highly correlated with proximity to the OBC class and the synthesis DSL. The OBC property requires values to be generated top-down: local constraints are easy to express, while long-distance dependencies across different parts of a value are not. The synthesis DSL and skeleton seeding (Section 5.1.2) reflect this bias, and parent annotation adds local Markov-style context to each choice. The results follow this boundary closely: where the class can express an invariant, programs learn it from relatively few examples and maintain it better than matched-n transformers. The *structure* columns of Table 2 (novel violation, precision) give the result. The clearest example is the V2 row, containing sorted lists, search trees, and scoped variables. When learning interval invariants, for instance, the transformer must infer these constraints from data alone, while the program can encode them directly. Programs violate constraints on 13–20% of novel samples, substantially lower than the untrained and random-pretrained transformers. On the other hand, the synthesis generally fails to learn constraints which are difficult for OBC programs or the synthesis DSL to express. On V1 and V3, for instance, the program loses to both transformers at $n = 30$ and higher: V3 cannot easily be expressed top-down, and the V1 program that threads a value through the recursion is rarely found.

6.4 Learning Programs over Real-World Data

Synthesizing probabilistic programs which fit real-world data is a more challenging learning task. Of course, we cannot expect programs to fit these distributions perfectly. Instead, we evaluate the recovery of structural constraints present in the dataset, focusing on syntactic and semantic validity of the trees representing the regexes and SPARQL queries. We find that synthesized programs achieve reasonable learning performance in certain respects, though they are not especially competitive as likelihood estimators. On the regex dataset, for instance, the program is roughly level on fit with the transformers at $n = 100$, though it is generally behind at $n = 300$. (It should be noted that program fit on regexes is dominated by a lexical task, predicting the individual characters of fixed strings. When characters within a class are treated uniformly, the synthesized program actually beats the untrained transformer at $n = 100$ on 67% of seeds, and on the bounded-repeat subset, where structure recovery is paramount, it beats both transformers at every $n \geq 30$, sometimes uniformly across seeds.)

For regexes, the program's value lies chiefly in the structure it supplies as a teacher (see Section 6.5) and in the validity of its samples, more than as an accurate likelihood estimator. Programs produce compiling regexes in 51–54% of samples at every n, against 0.3–5% for the untrained transformer and 10–24% for the random-pretrained one. The pattern of good local constraint recovery we note on the synthetic grid continues on the regex set: local rules, such as bounded repeat or character range orderings, are almost always respected. At $n = 300$, for instance, only 0.5% of bounded repeats `{m,n}` in program samples have $m > n$, against 9.3% and 30.7% for the transformers. Furthermore, only 0.5% of character ranges are reversed (e.g. `[z-a]`). On the other hand, long-range dependencies are typically not recovered; 77% of backreferences in program samples name an unclosed group compared to 25–39% for the transformers.

Programs fit to the SPARQL dataset also function better as structural teachers than as accurate likelihood estimators, though in that role it functions better than the baselines. Indeed, the program is the best density model at $n = 30$ and $n = 100$ (Figure 5). At $n = 300$ it still beats the untrained

		excess over truth			fit wins			novel violation			precision wins	
Block	n	P	R	N	vs. R	vs. N	recall	P	R	N	vs. R	vs. N
S1–S3	10	**0.35**	0.38	2.58	.43	.67	.893	0.03	**0.02**	0.33	.34	.92
	30	**0.16**	0.29	1.23	.56	.77	.970	0.02	**0.02**	0.19	.36	.88
	100	**0.03**	0.21	0.56	.82	.93	.995	**0.01**	0.02	0.09	.46	.79
	300	**0.01**	0.12	0.25	.91	.97	1	**0.01**	0.02	0.06	.47	.74
S4	10	**0.93**	1.12	3.89	.43	.62	.904	**0.20**	0.30	0.43	.66	.75
	30	**0.55**	0.92	2.01	.62	.71	.975	**0.22**	0.33	0.34	.72	.66
	100	**0.20**	0.71	1.10	.79	.77	.997	**0.22**	0.36	0.27	.74	.65
	300	**0.13**	0.40	0.58	.74	.72	1	0.28	0.33	**0.22**	.59	.48
V2	10	**0.41**	1.05	3.77	.59	.87	.909	**0.15**	0.35	0.47	.83	.86
	30	**0.44**	0.89	2.22	.73	.92	.965	**0.14**	0.35	0.39	.87	.84
	100	**0.29**	0.81	1.31	.89	.93	.992	**0.13**	0.36	0.34	.89	.84
	300	**0.21**	0.51	0.75	.93	.90	.998	**0.20**	0.42	0.34	.81	.72
V1	10	**0.49**	1.37	3.45	.56	.74	.877	**0.54**	0.60	0.62	.49	.54
	30	1.15	**0.93**	2.17	.38	.61	.958	0.67	0.57	**0.49**	.35	.28
	100	1.26	**0.67**	1.08	.14	.34	.994	0.80	0.39	**0.35**	.07	.03
	300	1.22	**0.46**	0.64	.03	.11	.999	0.87	0.32	**0.31**	.01	.01
V3	10	**2.32**	3.08	5.37	.63	.88	.933	**0.71**	0.72	0.72	.57	.40
	30	**2.10**	2.74	3.14	.65	.72	.982	0.76	**0.75**	0.75	.44	.34
	100	1.99	2.16	**1.77**	.57	.36	.995	0.83	0.81	**0.64**	.49	.23
	300	1.93	1.26	**0.64**	.39	.32	.999	0.89	0.77	**0.56**	.26	.19

Table 2. The synthetic grid, by block and training-set size n, for the synthesized program (P), the random-pretrained transformer (R), and the untrained transformer (N). *Fit:* median excess NLL over the ground truth (nats per tree), over the test trees the program covers for P and over all test trees for R and N; the fraction of runs on which the program fits better than each transformer, paired on the trees it covers; and the program's mean support recall. *Structure:* mean rate at which novel, well-formed samples fall outside the ground truth's support; and the fraction of runs on which the program's novel samples do so less often than each transformer's (ties count as losses). The best excess and violation rate in each row are in bold; win fractions above .5 favor the program.

transformer on every seed, but its relative advantage diminishes, as random pretraining overtakes it slightly.

As with the regex dataset, the structural recovery by synthesized programs is much stronger than the baselines on the SPARQL dataset. Across all seeds, 40–48% of program samples are valid queries, against 0–2% for no-pretraining and 3–21% for random-pretraining. The program is more valid than both transformers on every seed at every n, and already at full strength at $n = 10$. Similarly to regex, long-range SPARQL dependencies are often not recovered. Synthesized programs break SPARQL grouping rules, `BIND` scope restrictions, `AS` scope and variable ordering constraints in up to 99% of applicable samples. The full-pool transformer learns all of these rules (at worst 12% violations), indicating the failure reflects the program class rather than the data.

Parent annotation provides most of the benefit that programs enjoy on real data. Without it, only 2–3% of program samples are valid SPARQL queries, because the root query pattern is drawn from the same rule as every nested one. Annotation also has a cost on the grid: it gives up the exact V2 recovery of the unannotated program in exchange for better fit.

6.5 Data Augmentation with Synthesized Programs

In this section we evaluate whether the data-efficient summaries we learn can serve as effective teachers for downstream learning tasks. We do this by comparing the performance of the program-pretrained transformers against other transformers that were not fine-tuned on synthetically generated data from our learned programs.A synthesized program is a sampler, so it can amplify

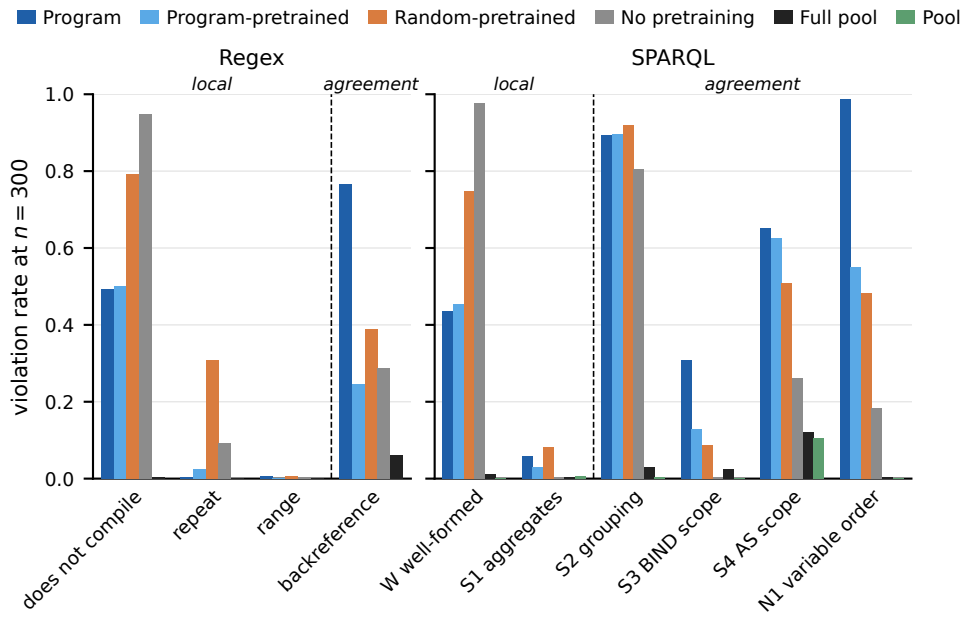


Fig. 6. Fraction of samples violating each validity rule at $n = 300$ (mean over seeds; lower is better), grouped into *local* rules (placement and order of one construct) and *agreement* rules (relating two uses of a variable or group). *Regex:* fails `re.compile`; repeat `{m,n}` with $m > n$; set range with reversed bounds; backreference to an unclosed group. *SPARQL:* not printable as a query (W); misplaced or nested aggregate (S1); aggregate query projects a non-`GROUP BY` variable (S2); `BIND` to an in-scope variable (S3); `AS ?v` rebinds a bound variable (S4); variables not numbered in first-use order (N1). Except for compilation and W, rates are among samples containing the rule's construct. Pool: the SPARQL training data, which includes some queries the specification forbids.

what it learns from only n trees into an arbitrarily large corpus. We find that pretraining on the program-sampled corpus is in fact the best of the three transformer pretraining strategies on both real-world domains at every $n \geq 30$ (Figure 7, left). The program-pretrained transformer beats the random-pretrained one on every seed at $n \geq 30$ on SPARQL and at $n \geq 100$ on regex. The advantage comes from the program itself, rather than from pretraining in general: for instance, only 12% of the samples from the random-pretrained regex model actually compile, against 46% of program-pretrained samples.

Notably, the program-pretrained transformer fits the trees the program covers better than the program itself on every run in both real-world domains (Figure 7, right). At $n = 300$, the program-pretrained model is a better-fit model of the SPARQL dataset on all 30 seeds, and by a substantial margin (a median of 17.5 nats). The program reinforces the structural information which is encoded the student model by introducing novel samples which reify the constraints present in the training set, and fine-tuning locally corrects some of the program's residual. The full-pool transformer, trained on over 100,000 trees, remains a much better model, but at only 300 training trees, program pretraining closes 41% of the gap between the untrained transformer and the reference on SPARQL, and 14% on regex.

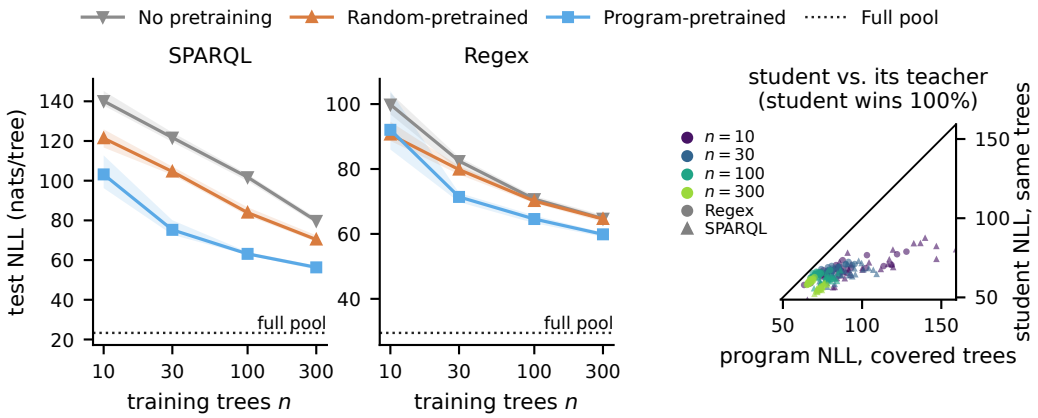


Fig. 7. Left: Test NLL for transformers trained without pretraining, pretrained on random well-typed values, or pretrained on samples from a synthesized program, with 10–300 training trees (median and interquartile range across seeds); the dotted line shows a transformer trained on the full pool. Right: For each run on both domains, the synthesized program's NLL on the test trees it covers versus the NLL of the transformer pretrained on samples from that program. Every point lies below the diagonal: on the covered trees, the student has lower NLL than its teacher.

6.6 Discussion and Limitations

Within the OBC class, synthesized programs recover the structure of a distribution from tens to hundreds of examples: they keep its invariants more reliably than transformers trained on the same data (RQ2), fit it more closely on the grid and on SPARQL up to $n = 100$, (RQ1), and transfer that structure to a neural model through pretraining (RQ3). Outside the class, on agreement rules and aggregate values, they fail predictably, and the full-pool transformer shows that these failures reflect the class rather than the data. Below $n = 30$, programs omit constructs absent from the training set, so they are useful there as pretraining sources and for their validity, but not as stand-alone density models. We believe that with better program synthesis techniques, a more expressive synthesis DSL, and perhaps more advanced methods such as ensemble learning or specification mining, some of the gaps on real-world data could be closed, especially in the data augmentation setting.

No transformer uses constrained decoding [18], which biases their real-data violation rates upward, since malformed outputs count as invalid; on the grid, we report precision on well-formed samples only, so those results are unaffected. We believe the choice is justified: with enough data, malformed outputs nearly disappear (99.6% of the full-pool transformer's regex samples compile, and 96.6% of its SPARQL samples are valid), and constrained decoding would make the likelihoods a model assigns correspond to a different distribution from the one it samples.

7 RELATED WORK

Operational Profiling. Operational profiling [20, 39] is an approach to software evaluation which emphasizes testing and measuring software systems within the context they are designed to be used or are currently deployed. Operational profiling involves both quantitative and statistical methods to characterize typical usage [15, 26], as well as qualitative and user-centric methods to develop user stories and contexts to improve software reliability [2, 43, 51]. Operational profiling seeks to understand empirically how software behaves when situated in its operational context,

and subsequently test software in that context. In this vein, Pavese et al. [45] study the problem of learning PCFGs over real-world program inputs for use in robustness testing. Our technique is similar to this approach but uses a more expressive hypothesis space.

Probabilistic Program Synthesis. Prior work has explored techniques to synthesize probabilistic programs which fit datasets or satisfy specifications. Some existing approaches use MCMC methods to synthesize programs in various DSLs which maximize the likelihood of an observed dataset [42, 49]. DreamCoder [16] is another synthesis-driven learning tool which learns representations using programs. Learning programs as representations is a standard inductive bias in machine learning [38]. We frame our tool as as a framework which learns a distribution over tree-structured values using probabilistic program synthesis; as a representation-learning approach, we produce a program as our representation.

Probabilistic Programming Languages. Probabilistic programming languages have long been a foundational tool in generative modeling contexts [11, 19, 23, 24]. While probabilistic programming is a powerful computational paradigm, inference is often inefficient or intractable. Recent research has made great strides in efficient discrete probabilistic inference. Languages such as `Dice` [27] exploit knowledge compilation to transform discrete probabilistic programs into binary decision diagrams, allowing efficient inference on a wide variety of programs. Loaded Dice [56] builds upon Dice by adding automatic differentiation on the computation graph induced by the BDD representation of a probabilistic program for use in parameter optimization. We take a very similar gradient-optimization approach for our parameter learning mode, though Loaded Dice is more focused on optimizing synthetic objectives such as dataset entropy.

Previous work has explored exact inference on recursive probabilistic programs [8, 9, 35, 54]. Many probabilistic programming languages use lazy strategies to achieve fast marginalization and inference [8, 31, 41, 46, 50]. Languages such as Pluck [8] which support recursive programs over inductive datatypes exploit the fact that expressions can naturally be evaluated to *weak-head normal form*, where the forced constructor can be thought of as an observation which prunes the value space to be enumerated. Our guarded recursion scheme forces recursive calls to only occur where the result of a recursive call will be the child of a constructor in WHNF, meaning that recursive calls can always be thunked.

The exactness guarantee of OBC differentiates our work from existing lazy inference techniques, which are sound but not necessarily convergent. Figaro's lazy factored inference [46], for example, returns upper and lower bounds on probabilities which are not guaranteed to converge to a single value. Pluck [8] dynamically unrolls recursive programs under its lazy inference procedure, but does not guarantee that exact inference bounds the number of probabilistic choices. Indeed, Bowers et al. [8] note that "[i]t would be interesting to explore whether there are sensible syntactic restrictions that make such programs inexpressible;" we partially answer this challenge.

Modal Types and Guarded Recursion. Modal types for recursion were first described by Nakano [40], have been used to prove productivity in corecursive programs [3]. Guarded recursion schemes have been applied to tree-like structures [6], and have been used to reason about probabilistic computation and non-determinism [52], as well as almost-sure termination [25]. Our approach draws from this work in order to prove an eventual stabilization property (OBC) about unrolled probabilistic programs.

REFERENCES

[1] Patricia A. Apellaniz, Juan Parras, and Santiago Zazo. 2024. An Improved Tabular Data Generator with VAE-GMM Integration. In *2024 32nd European Signal Processing Conference (EUSIPCO)*. 1886–1890. https://doi.org/10.23919/EUSIPCO63174.2024.10715230

[2] S. Arora and R.B. Misra. 2005. Software Reliability Improvement through Operational Profile Testing. In *Annual Reliability and Maintainability Symposium, 2005. Proceedings.* 621–627. https://doi.org/10.1109/RAMS.2005.1408433

[3] Robert Atkey and Conor McBride. 2013. Productive Coprogramming with Guarded Recursion. In *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*. ACM, Boston Massachusetts USA, 197–208. https://doi.org/10.1145/2500365.2500597

[4] Hugo Barbalho, Patricia Kovaleski, Beibin Li, Luke Marshall, Marco Molinaro, Abhisek Pan, Eli Cortez, Matheus Leao, Harsh Patwari, Zuzu Tang, Tamires Santos, Larissa Rozales Gonçalves, David Dion, Thomas Moscibroda, and Ishai Menache. 2023. Virtual Machine Allocation with Lifetime Predictions. In *MLSys*.

[5] Matthew J Beal and Zoubin Ghahramani. 2003. The Variational Bayesian EM Algorithm for Incomplete Data: With Application to Scoring Graphical Model Structures. In *Bayesian Statistics 7*, V Lindley, J M Bernardo, M J Bayarri, J O Berger, A P Dawid, D Heckerman, A Fm Smith, and M West (Eds.). Oxford University PressOxford, 453–463. https://doi.org/10.1093/oso/9780198526155.003.0025

[6] Lars Birkedal, Rasmus Ejlers Mogelberg, Jan Schwinghammer, and Kristian Stovring. 2011. First Steps in Synthetic Guarded Domain Theory: Step-Indexing in the Topos of Trees. In *2011 IEEE 26th Annual Symposium on Logic in Computer Science*. 55–64. https://doi.org/10.1109/LICS.2011.16

[7] David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. 2017. Variational Inference: A Review for Statisticians. *J. Amer. Statist. Assoc.* 112, 518 (April 2017), 859–877. https://doi.org/10.1080/01621459.2017.1285773

[8] Maddy Bowers, Alexander K. Lew, Joshua B. Tenenbaum, Armando Solar-Lezama, and Vikash K. Mansinghka. 2025. Stochastic Lazy Knowledge Compilation for Inference in Discrete Probabilistic Programs. *Proceedings of the ACM on Programming Languages* 9, PLDI (June 2025), 1863–1887. https://doi.org/10.1145/3729325

[9] David Chiang, Colin McDonald, and Chung-chieh Shan. 2023. Exact Recursive Probabilistic Programming. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (April 2023), 665–695. https://doi.org/10.1145/3586050

[10] Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. Association for Computing Machinery, New York, NY, USA, 268–279. https://doi.org/10.1145/351240.351266

[11] Guillaume Claret, Sriram K. Rajamani, Aditya V. Nori, Andrew D. Gordon, and Johannes Borgström. 2013. Bayesian Inference Using Data Flow Analysis. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. ACM, Saint Petersburg Russia, 92–102. https://doi.org/10.1145/2491411.2491423

[12] Swaraj Dash, Younesse Kaddar, Hugo Paquet, and Sam Staton. 2023. Affine Monads and Lazy Structures for Bayesian Programming. *LazyPPL: Affine monads and lazy structures for Bayesian programming* 7, POPL (Jan. 2023), 46:1338–46:1368. https://doi.org/10.1145/3571239

[13] James C. Davis, Louis G. Michael Iv, Christy A. Coghlan, Francisco Servant, and Dongyoon Lee. 2019. Why Aren't Regular Expressions a Lingua Franca? An Empirical Study on the Re-Use and Portability of Regular Expressions. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Tallinn Estonia, 443–454. https://doi.org/10.1145/3338906.3338909

[14] Yihe Dong, Piotr Indyk, Ilya Razenshteyn, and Tal Wagner. 2020. Learning Space Partitions for Nearest Neighbor Search. In *Eighth International Conference on Learning Representations (ICLR)*.

[15] L. du Bousquet, F. Ouabdesselam, and J.-L. Richier. 1998. Expressing and Implementing Operational Profiles for Reactive Software Validation. In *Proceedings Ninth International Symposium on Software Reliability Engineering (Cat. No.98TB100257)*. 222–230. https://doi.org/10.1109/ISSRE.1998.730885

[16] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, Virtual Canada, 835–850. https://doi.org/10.1145/3453483.3454080

[17] Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis* (0 ed.). Chapman and Hall/CRC. https://doi.org/10.1201/b16018

[18] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 10932–10952. https://doi.org/10.18653/v1/2023.emnlp-main.674

[19] Zoubin Ghahramani. 2015. Probabilistic Machine Learning and Artificial Intelligence. *Nature* 521, 7553 (May 2015), 452–459. https://doi.org/10.1038/nature14541

[20] M. Gittens, H. Lutfiyya, and M. Bauer. 2004. An Extended Operational Profile Model. In *15th International Symposium on Software Reliability Engineering*. 314–325. https://doi.org/10.1109/ISSRE.2004.8

[21] Harrison Goldstein, Samantha Frohlich, Meng Wang, and Benjamin C. Pierce. 2023. Reflecting on Random Generation. *Proceedings of the ACM on Programming Languages* 7, ICFP (Aug. 2023), 322–355. https://doi.org/10.1145/3607842

[22] Harrison Goldstein and Benjamin C. Pierce. 2022. Parsing Randomness. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (Oct. 2022), 89–113. https://doi.org/10.1145/3563291

[23] Noah Goodman, Vikash Mansinghka, Daniel M. Roy, Keith Bonawitz, and Joshua B. Tenenbaum. 2014. Church: A Language for Generative Models. https://doi.org/10.48550/arXiv.1206.3255 arXiv:1206.3255 [cs]

[24] Andrew D. Gordon, Thomas A. Henzinger, Aditya V. Nori, and Sriram K. Rajamani. 2014. Probabilistic Programming. In *Future of Software Engineering Proceedings (FOSE 2014)*. Association for Computing Machinery, New York, NY, USA, 167–181. https://doi.org/10.1145/2593882.2593900

[25] Simon Oddershede Gregersen, Alejandro Aguirre, Philipp G. Haselwarter, Joseph Tassarotti, and Lars Birkedal. 2024. Almost-Sure Termination by Guarded Refinement. *Proceedings of the ACM on Programming Languages* 8, ICFP (Aug. 2024), 203–233. https://doi.org/10.1145/3674632

[26] Herman Hartmann. 2016. A Statistical Analysis of Operational Profile Driven Testing. In *2016 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. 109–116. https://doi.org/10.1109/QRS-C.2016.18

[27] Steven Holtzen, Guy Van Den Broeck, and Todd Millstein. 2020. Scaling Exact Inference for Discrete Probabilistic Programs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (Nov. 2020), 1–31. https://doi.org/10.1145/3428208

[28] Irvin Hwang, Andreas Stuhlmüller, and Noah D. Goodman. 2011. Inducing Probabilistic Programs by Bayesian Program Merging. https://doi.org/10.48550/arXiv.1110.5667 arXiv:1110.5667 [cs]

[29] Mark Johnson. 1998. PCFG Models of Linguistic Tree Representations. *Computational Linguistics* 24, 4 (1998), 613–632.

[30] Diederik P. Kingma and Jimmy Ba. 2017. Adam: A Method for Stochastic Optimization. https://doi.org/10.48550/arXiv.1412.6980 arXiv:1412.6980 [cs.LG]

[31] Oleg Kiselyov and Chung-chieh Shan. 2009. Embedded Probabilistic Programming. In *Domain-Specific Languages*, Walid Mohamed Taha (Ed.). Vol. 5658. Springer Berlin Heidelberg, Berlin, Heidelberg, 360–384. https://doi.org/10.1007/978-3-642-03034-5_17

[32] Daphne Koller, David McAllester, and Avi Pfeffer. 1997. Effective Bayesian Inference for Stochastic Programs. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence and Ninth Conference on Innovative Applications of Artificial Intelligence (AAAI'97/IAAI'97)*. AAAI Press, Providence, Rhode Island, 740–747.

[33] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 489–504. https://doi.org/10.1145/3183713.3196909

[34] Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum. 2015. Human-Level Concept Learning through Probabilistic Program Induction. *Science* 350, 6266 (Dec. 2015), 1332–1338. https://doi.org/10.1126/science.aab3050

[35] Jianlin Li, Eric Wang, and Yizhou Zhang. 2024. Compiling Probabilistic Programs for Variable Elimination with Information Flow. *Proceedings of the ACM on Programming Languages* 8, PLDI (June 2024), 1755–1780. https://doi.org/10.1145/3656448

[36] Dong C. Liu and Jorge Nocedal. 1989. On the Limited Memory BFGS Method for Large Scale Optimization. *Mathematical Programming* 45, 1 (Aug. 1989), 503–528. https://doi.org/10.1007/BF01589116

[37] Stanislav Malyshev, Markus Krötzsch, Larry González, Julius Gonsior, and Adrian Bielefeldt. 2018. Getting the Most out of Wikidata: Semantic Technology Usage in Wikipedia's Knowledge Graph. In *Proceedings of the 17th International Semantic Web Conference (ISWC'18) (LNCS, Vol. 11137)*, Denny Vrandečić, Kalina Bontcheva, Mari Carmen Suárez-Figueroa, Valentina Presutti, Irene Celino, Marta Sabou, Lucie-Aimée Kaffee, and Elena Simperl (Eds.). Springer, 376–394.

[38] Stephen Muggleton and Luc De Raedt. 1994. Inductive Logic Programming: Theory and Methods. *The Journal of Logic Programming* 19–20 (May 1994), 629–679. https://doi.org/10.1016/0743-1066(94)90035-3

[39] J.D. Musa. 1993. Operational Profiles in Software-Reliability Engineering. *IEEE Software* 10, 2 (March 1993), 14–32. https://doi.org/10.1109/52.199724

[40] H. Nakano. 2000. A Modality for Recursion. In *Proceedings Fifteenth Annual IEEE Symposium on Logic in Computer Science (Cat. No.99CB36332)*. 255–266. https://doi.org/10.1109/LICS.2000.855774

[41] Praveen Narayanan, Jacques Carette, Wren Romano, Chung-chieh Shan, and Robert Zinkov. 2016. Probabilistic inference by program transformation in Hakaru (system description). In *International Symposium on Functional and Logic Programming - 13th International Symposium, FLOPS 2016, Kochi, Japan, March 4-6, 2016, Proceedings*. Springer, 62–79. https://doi.org/10.1007/978-3-319-29604-3_5

[42] Aditya V. Nori, Sherjil Ozair, Sriram K. Rajamani, and Deepak Vijaykeerthy. 2015. Efficient Synthesis of Probabilistic Programs. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Portland OR USA, 208–217. https://doi.org/10.1145/2737924.2737982

[43] Süleyman Özekici Özekici, İ. Kuban Altínel, and Selma Özçelikyürek. 2000. Testing of Software with an Operational Profile. *Naval Research Logistics* 47, 8 (Dec. 2000), 620–634. https://doi.org/10.1002/1520-6750(200012)47:8<620::AID-

NAV2>3.0.CO;2-V

[44] Thorsten Papenbrock, Jens Ehrlich, Jannik Marten, Tommy Neubert, Jan-Peer Rudolph, Martin Schönberg, Jakob Zwiener, and Felix Naumann. 2015. Functional Dependency Discovery: An Experimental Evaluation of Seven Algorithms. *Proceedings of the VLDB Endowment* 8, 10 (2015), 1082–1093.

[45] Esteban Pavese, Ezekiel Soremekun, Nikolas Havrikov, Lars Grunske, and Andreas Zeller. 2018. Inputs from Hell: Generating Uncommon Inputs from Common Samples. https://doi.org/10.48550/arXiv.1812.07525 arXiv:1812.07525 [cs.SE]

[46] Avi Pfeffer, Brian E. Ruttenberg, Amy Lynn Sliva, Michael Howard, and Glenn Takata. 2015. Lazy Factored Inference for Functional Probabilistic Programming. *ArXiv* abs/1509.03564 (2015). https://api.semanticscholar.org/CorpusID:1257469

[47] Benjamin C. Pierce (Ed.). 2005. *Advanced Topics in Types and Programming Languages*. MIT Press, Cambridge, Mass.

[48] Dan Roth. 1996. On the Hardness of Approximate Reasoning. *Artificial Intelligence* 82, 1-2 (April 1996), 273–302. https://doi.org/10.1016/0004-3702(94)00092-1

[49] Feras A. Saad, Marco F. Cusumano-Towner, Ulrich Schaechtle, Martin C. Rinard, and Vikash K. Mansinghka. 2019. Bayesian Synthesis of Probabilistic Programs for Automatic Data Modeling. *Proceedings of the ACM on Programming Languages* 3, POPL (Jan. 2019), 1–32. https://doi.org/10.1145/3290350 arXiv:1907.06249 [cs]

[50] Adam Ścibior, Ohad Kammar, and Zoubin Ghahramani. 2018. Functional Programming for Modular Bayesian Inference. *Proceedings of the ACM on Programming Languages* 2, ICFP (July 2018), 1–29. https://doi.org/10.1145/3236778

[51] Carol Smidts, Chetan Mutha, Manuel Rodríguez, and Matthew J. Gerber. 2014. Software Testing with an Operational Profile: OP Definition. *ACM Comput. Surv.* 46, 3 (Feb. 2014), 39:1–39:39. https://doi.org/10.1145/2518106

[52] Philipp Stassen, Rasmus Ejlers Møgelberg, Maaike Annebet Zwart, Alejandro Aguirre, and Lars Birkedal. 2025. Modelling Recursion and Probabilistic Choice in Guarded Type Theory. *Proceedings of the ACM on Programming Languages* 9, POPL (Jan. 2025), 1417–1445. https://doi.org/10.1145/3704884

[53] Andreas Stolcke and Stephen Omohundro. 1994. Inducing Probabilistic Grammars by Bayesian Model Merging. In *Grammatical Inference and Applications*, Rafael C. Carrasco and Jose Oncina (Eds.). Springer, Berlin, Heidelberg, 106–118. https://doi.org/10.1007/3-540-58473-0_141

[54] Andreas Stuhlmüller and Noah D. Goodman. 2012. A Dynamic Programming Algorithm for Inference in Recursive Probabilistic Programs. https://doi.org/10.48550/arXiv.1206.3555 arXiv:1206.3555 [cs.AI]

[55] Syed Tazwar, Max Knobbout, Enrique Quesada, and Mirela Popa. 2024. Tab-VAE: A Novel VAE for Generating Synthetic Tabular Data:. In *Proceedings of the 13th International Conference on Pattern Recognition Applications and Methods*. SCITEPRESS - Science and Technology Publications, Rome, Italy, 17–26. https://doi.org/10.5220/0012302400003654

[56] Ryan Tjoa, Poorva Garg, Harrison Goldstein, Todd Millstein, Benjamin C. Pierce, and Guy Van Den Broeck. 2025. Tuning Random Generators: Property-Based Testing as Probabilistic Programming. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (Oct. 2025), 894–921. https://doi.org/10.1145/3763082

[57] Jan-Willem van de Meent, Brooks Paige, Hongseok Yang, and Frank Wood. 2021. An Introduction to Probabilistic Programming. https://doi.org/10.48550/arXiv.1809.10756 arXiv:1809.10756 [stat]

[58] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc.

[59] P.J. Werbos. 1990. Backpropagation through Time: What It Does and How to Do It. *Proc. IEEE* 78, 10 (Oct. 1990), 1550–1560. https://doi.org/10.1109/5.58337

[60] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. 2019. Modeling Tabular Data Using Conditional GAN. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Number 659. Curran Associates Inc., Red Hook, NY, USA, 7335–7345.

A PROBABILISTIC SEMANTICS

A.1 Proof of Theorem 4.2

We prove the result by structural induction on the denotation. For base values, variable references, and sampling operations, the denotations are point masses. A constructor application matches only the correct constructor and distributes its mass pointwise across its arguments. Binding forms such as let-bindings and function applications marginalize the distributions of their arguments over the bound variables. The equation for case destructuring is the only one with an additional case: its final summand $\mu[\![e]\!]_{\rho,\phi}(\bot)$ is the only term (apart from the function-context approximants described below) that places mass on $\bot$.

A.2 Function fixed points and approximants

The zeroth approximant of the function context ϕ_0 is a point mass on $\bot$, rather than the zero distribution $\phi_0(f)(\overline{\delta})(\delta') = 0$. This is the appropriate choice in this context. To see why, consider the gen() example on page 7. The zero distribution would assign zero probability to every finite list at every approximant ϕ_k, even though ϕ_k represents gen unrolled k times. Starting at $\bot$ instead allows ϕ_k to move its probability mass to lists of length k, as intended.

A.3 Proof sketch of Theorem 4.3

We build the coupling by structural induction on $\mu[\![\cdot]\!]_{\rho,\phi}$. Monotonicity in ρ is needed to carry the induction through case , because the environment is extended with variables bound to the scrutinee's children, and the refined value of a child may be above its coarse value.

A.4 Proof of Theorem 4.5

The proof has two parts. The first is a semantic reading of the modal index: an expression typed at $\triangleright^n A$, evaluated under the k-th approximant, produces partial values whose undefined parts all lie at depth at least $k - n$, up to a constant that accounts for the finitely many non-recursive call layers. The second observes that once an approximant is defined below the height of a complete observation, no further unrolling can move mass onto or off of that observation.

Notation. Throughout, fix a well-typed program

$$P = \text{define } f_1(\overline{x_1 : \tau_1}) : \tau_1' \coloneqq e_1;\ \ldots\ \text{define } f_m(\overline{x_m : \tau_m}) : \tau_m' \coloneqq e_m;\ e_{m+1},$$

where we write e_{m+1} for the main body, and let Σ be the signature accumulated by T-Defn. By T-Defn, the body e_i of f_i is typed against a signature containing exactly $f_1, \ldots, f_i$, so every call in e_i is to some f_j with $j \le i$, and the call is a self-call exactly when $j = i$. We say that e_i has *position* i, and that the main body has position $m + 1$.

For a partial value $\delta \in \mathcal{V}_\bot$, define its *definedness depth* $\mathrm{dp}(\delta) \in \mathbb{N} \cup \{\infty\}$ as the depth of its shallowest $\bot$, or ∞ if it has none:

$$\mathrm{dp}(\bot) = 0, \qquad \mathrm{dp}(C(\delta_1, \ldots, \delta_r)) = 1 + \min_{i \le r} \mathrm{dp}(\delta_i),$$

with the convention that the minimum over an empty sequence is ∞. Thus, nullary constructors and, inductively, all complete values have infinite definedness depth. For a complete value $v \in \mathcal{V}$, define its *height* $\mathrm{h}(v)$ by $\mathrm{h}(C) = 1$ and $\mathrm{h}(C(v_1, \ldots, v_r)) = 1 + \max_i \mathrm{h}(v_i)$. Both functions are computable. We will need the following elementary relationship between them.

Lemma A.1 (Defined below the observation)**.** *Let $v \in \mathcal{V}$ and $\delta \in \mathcal{V}_\bot$ with $\delta \sqsubseteq v$. If $\mathrm{dp}(\delta) \ge \mathrm{h}(v)$, then $\delta = v$.*

Proof. By induction on v. Since $\mathrm{h}(v) \geq 1$, $\delta \neq \bot$, so $\delta = C(\overline{\delta})$ and $v = C(\overline{v})$ for the same constructor with $\delta_i \sqsubseteq v_i$ for each i. Then $\mathrm{dp}(\delta_i) \geq \mathrm{dp}(\delta) - 1 \geq \mathrm{h}(v) - 1 \geq \mathrm{h}(v_i)$, so $\delta_i = v_i$ by the induction hypothesis. □

Levels. The modal index is a *relative* quantity, and the semantics of Figure 3 charges one approximant per call, whether or not the call is recursive. To relate these quantities, we introduce a level $\ell \in \mathbb{Z}$ that decreases with each call. Because the non-self call graph is acyclic and ordered by position, a body at position i evaluated under ϕ_k has at most $i - 1$ non-recursive calls stacked beneath it. We will show that the level $\ell = k - i + 1$ is always available to it. Negative levels are permitted and make every claim below vacuous, which is what allows the induction to start at ϕ_0.

For a context Γ and an environment ρ defined on $\mathrm{dom}(\Gamma)$, write $\rho \models_\ell \Gamma$ when $\mathrm{dp}(\rho(x)) \geq \ell - n$ for every $x : \triangleright^n A \in \Gamma$.

Lemma A.2 (Semantic typing)**.** *Let $\Sigma; \Gamma; f \vdash e : \triangleright^n A$ be derivable, where e occurs in the body at position i (so $f = f_i$, or $f = \cdot$ and $i = m + 1$). Then for every $k \geq 0$, every level $\ell \leq k - i + 1$, and every $\rho \models_\ell \Gamma$, every δ in the support of $\mu[\![e]\!]_{\rho,\phi_k}$ satisfies $\mathrm{dp}(\delta) \geq \ell - n$.*

Proof. By induction on k, and for each k by an inner induction on the typing derivation. We use the outer induction hypothesis only for calls, whose callee bodies are evaluated under ϕ_{k-1}. For all other subterms, which are evaluated under ϕ_k, we use the inner induction hypothesis. Fix k, $\ell \leq k - i + 1$, and $\rho \models_\ell \Gamma$.

T-Var. The support is $\{\rho(x)\}$, and $\mathrm{dp}(\rho(x)) \geq \ell - n$ by $\rho \models_\ell \Gamma$.

T-Flip, T-Geom, T-Constr-Null. The support consists of complete values, whose definedness depth is ∞.

T-Later-Sub. The inner hypothesis gives $\mathrm{dp}(\delta) \geq \ell - m \geq \ell - n$ since $m \leq n$.

T-Constr. The support of $\mu[\![C(\overline{e})]\!]_{\rho,\phi_k}$ consists of values $C(\overline{\delta})$ with each δ_r in the support of $\mu[\![e_r]\!]_{\rho,\phi_k}$. Each e_r is typed at $\triangleright^{n+1} A_r$, so $\mathrm{dp}(\delta_r) \geq \ell - n - 1$ by the inner hypothesis, and hence $\mathrm{dp}(C(\overline{\delta})) = 1 + \min_r \mathrm{dp}(\delta_r) \geq \ell - n$. This is the step that converts one layer of observable output into one unit of modal debt.

T-Let. Mass on δ arises from some δ' in the support of $\mu[\![e]\!]_{\rho,\phi_k}$ together with δ in the support of $\mu[\![e']\!]_{\rho[x \mapsto \delta'],\phi_k}$. If e is typed at $\triangleright^{n_1} A_1$ then $\mathrm{dp}(\delta') \geq \ell - n_1$ by the inner hypothesis, so $\rho[x \mapsto \delta'] \models_\ell \Gamma, x : \triangleright^{n_1} A_1$, and the inner hypothesis for e' gives $\mathrm{dp}(\delta) \geq \ell - n$.

T-Case. Let the scrutinee e be typed at $\triangleright^{n_0} A$ and the branches at $\triangleright^n A'$ with $n_0 \leq n$. By the equation for case in Figure 3, mass reaches δ in one of two ways. Either the scrutinee produces $C_i(\overline{\delta'})$ and branch i produces δ under $\rho[\overline{x_i} \mapsto \overline{\delta'}]$; then $\mathrm{dp}(C_i(\overline{\delta'})) \geq \ell - n_0$ by the inner hypothesis, so each $\mathrm{dp}(\delta'_r) \geq \ell - n_0 - 1$, the extended environment satisfies $\Gamma, \overline{x_i : \triangleright^{n_0+1} A_i}$ at level ℓ, and the inner hypothesis for the branch gives $\mathrm{dp}(\delta) \geq \ell - n$. Or the scrutinee produces $\bot$ and $\delta = \bot$; this requires $0 = \mathrm{dp}(\bot) \geq \ell - n_0$, so $\ell \leq n_0 \leq n$, and the claim $\mathrm{dp}(\bot) = 0 \geq \ell - n$ holds. This second case is the only place where we use the side condition $n_0 \leq n$: a branch may not promise to be more defined than a scrutinee that may still be undefined.

T-Call. Let the callee be $g = f_j$ with $j \leq i$ and $\Sigma(f_j) = (\triangleright^{m_1} A_1, \ldots, \triangleright^{m_r} A_r) \to \triangleright^{n_j} A'$, and let s be the shift chosen in the rule, so that each argument e_t is typed at $\triangleright^{m_t+s} A_t$ and the call at $\triangleright^{n_j+s} A'$. By the inner hypothesis, every δ'_t in the support of $\mu[\![e_t]\!]_{\rho,\phi_k}$ has $\mathrm{dp}(\delta'_t) \geq \ell - m_t - s = (\ell - s) - m_t$; that is, the argument environment $\rho_j = \{x_{jt} \mapsto \delta'_t\}$ satisfies $\rho_j \models_{\ell - s} \overline{x_j : \tau_j}$.

If $k = 0$, then $\phi_0(f_j)(\overline{\delta'})$ is the point mass on $\bot$, and $\mathrm{dp}(\bot) = 0 \geq \ell - n_j - s$ holds because $\ell \leq k - i + 1 = 1 - i \leq 0$.

If $k \geq 1$, then $\phi_k(f_j)(\overline{\delta'}) = \mu[\![e_j]\!]_{\rho_j,\phi_{k-1}}$, the body of f_j at position j evaluated under ϕ_{k-1}. We apply the outer induction hypothesis at $k - 1$ to the derivation $\Sigma; \overline{x_j : \tau_j}; f_j \vdash e_j : \triangleright^{n_j} A'$ supplied by T-Defn, at level $\ell - s$. This requires $\ell - s \leq (k - 1) - j + 1 = k - j$. If $j < i$ (a non-recursive

call), then $k - j \geq k - i + 1 \geq \ell \geq \ell - s$ since $s \geq 0$. If $j = i$ (a self-call), the rule requires $s \geq 1$, so $\ell - s \leq \ell - 1 \leq k - i = k - j$. In either case the hypothesis applies and yields $\mathrm{dp}(\delta) \geq (\ell - s) - n_j$ for every δ in the support of $\phi_k(f_j)(\overline{\delta'})$, which is the claim at $\triangleright^{n_j+s} A'$. Here the acyclicity of the non-self call graph is what pays for the $i-1$ slack in the level: a non-recursive callee sits at a strictly smaller position and so has one more approximant to spend, whereas a self-call must pay for its own approximant through $s \geq 1$. □

The quantification over all levels $\ell \leq k - i + 1$, rather than only the largest level, is what makes the T-Call case work for an arbitrary shift s: the callee's body is invoked at the lower level $\ell - s$, and the lemma must already hold there. This is the semantic counterpart of Lemma C.1.

Definedness bound. Applying Lemma A.2 to the premise of T-Prog-Main, with position $i = m+1$, $\rho = \varnothing$, and largest admissible level $\ell = k - m$, yields:

Corollary A.3 (Definedness of approximants)**.** *If $\cdot \vdash P : \triangleright^n A$, then for every k, every δ in the support of $\mu[\![e_{m+1}]\!]_{\varnothing,\phi_k}$ satisfies $\mathrm{dp}(\delta) \geq k - m - n$.*

In particular, taking $k \geq m + n + 1$ shows that $\mu[\![e_{m+1}]\!]_{\varnothing,\phi_k}(\bot) = 0$, which is Lemma B.1 for the main body; the same corollary applied to the body of each f_i at its own position gives Lemma B.1 for function bodies.

Stabilization. It remains to show that once an approximant is defined below the height of an observation, all later approximants agree with it on that observation.

Lemma A.4 (Stabilization)**.** *Let $v \in \mathcal{V}$ and $k \leq k'$, and suppose every δ in the support of $\mu[\![e_{m+1}]\!]_{\varnothing,\phi_k}$ satisfies $\mathrm{dp}(\delta) \geq \mathrm{h}(v)$. Then $[\![P]\!]_k(v) = [\![P]\!]_{k'}(v)$.*

Proof. Write μ_k and $\mu_{k'}$ for the two measures. Since v is complete, $\uparrow v = \{v\}$, so $[\![P]\!]_k(v) = \mu_k(v)$ and likewise for k'. By Theorem 4.3 there is a coupling γ of μ_k and $\mu_{k'}$ supported on pairs (δ, δ') with $\delta \sqsubseteq \delta'$. Then

$$\mu_{k'}(v) = \sum_{\delta} \gamma(\delta, v) = \sum_{\delta \sqsubseteq v,\ \mu_k(\delta) > 0} \gamma(\delta, v).$$

Every δ in this sum lies in the support of μ_k and satisfies $\delta \sqsubseteq v$, so by hypothesis and Lemma A.1, $\delta = v$. Hence $\mu_{k'}(v) = \gamma(v, v) \leq \mu_k(v)$. The reverse inequality $\mu_k(v) \leq \mu_{k'}(v)$ is the monotonicity statement of Theorem 4.3. □

Conclusion. Let $\cdot \vdash P : \triangleright^n A$ and define

$$\beta_P(v) = \mathrm{h}(v) + m + n,$$

which is total and computable. For $k \geq \beta_P(v)$, Corollary A.3 gives $\mathrm{dp}(\delta) \geq k - m - n \geq \mathrm{h}(v)$ for every value in the support of the k-th approximant. Thus, by Lemma A.4, $[\![P]\!]_k(v) = [\![P]\!]_{k'}(v)$ for every $k' \geq k$. Since $[\![P]\!](v) = \sup_{k'} [\![P]\!]_{k'}(v)$ by definition, and the sequence is constant from k onward, $[\![P]\!]_k(v) = [\![P]\!](v)$. This is Definition 4.4, and P is OBC.

Remarks. The bound β_P is linear in the height of the observation with unit slope: one approximant per layer, plus a constant. The constant m is an artifact of charging non-recursive calls an approximant each in Figure 3; defining Φ to inline non-recursive callees would remove it, at the cost of a more complicated functional. The constant n is the modal index of the main body, and is what permits main bodies such as head(gen()), whose first observable constructor is only available after the generator has been unrolled once. Nothing in the argument depends on Theorem B.3; finiteness of demand is what makes the inference procedure built on Theorem B.3 realizable as a circuit, but exactness at a finite approximant follows from definedness alone.

B PROBABILISTIC INFERENCE FOR TALI PROGRAMS

A well-typed Tali program has all of its recursive calls guarded by at least one constructor. One consequence is that every well-typed program eventually reduces to weak-head normal form (WHNF) with probability 1. In our semantics, this is represented as membership in a set of partial values $\uparrow C(\bot, \ldots, \bot)$.

Lemma B.1 (Weak-head normal form)**.** *Let e be an expression, let Σ be a well-formed function signature, and let Γ be a variable context defined on the free variables of e. Let ρ be a variable environment defined on the same variables as Γ. If $\Sigma; \Gamma; f \vdash e : \triangleright^n A$, then*

$$\lim_{k \to \infty} \mu[\![e]\!]_{\rho, \phi_k}(\bot) = 0,$$

or equivalently,

$$\sum_{C \in C_A} [\![e]\!]_\rho(C(\bot, \ldots, \bot)) = 1.$$

This holds even when e itself does not terminate. As mentioned in Section 4.2.2, probability mass can reach $\bot$ in two ways: by case-destructuring $\bot$ or by failing to produce a constructor. By T-Case, destructuring an unguarded self-call (i.e., a value at a modal type with index > 0) is allowed, but incurs a modal "debt" that can be repaid only by applying a constructor. Furthermore, because the function call graph excluding self-calls is acyclic, the number of steps needed to produce a constructor is bounded. Thus, some k exists such that the $(k+1)$-th unrolling produces a constructor. Consequently, we can marginalize over certain nonterminating executions, provided they place zero mass on $\bot$.

Lemma B.2 (Irrelevance)**.** *Let $l \in [0, 1]$, let ρ be a variable context, and let ϕ be a function context (either an approximant or the exact context). Let e_1 and e_2 be well-typed expressions. Suppose $[\![e_2]\!]_{\rho[x \mapsto \delta'], \phi}(\delta) = l$ for every partial value $\delta' \in \mathcal{V}_\bot$. Then*

$$[\![\text{let } x = e_1 \text{ in } e_2]\!]_{\rho, \phi}(\delta) = [\![e_2]\!]_{\rho[x \mapsto \bot], \phi}(\delta)$$

The proof is immediate: the constant l factors out of the marginalization, leaving the total mass $\sum_{\delta' \in \mathcal{V}_\bot} \mu[\![e_1]\!]_{\rho, \phi}(\delta')$, which is 1 by Theorem 4.2. This result also generalizes to other binding forms, including function application and case destructuring. There is no implicit conditioning on termination here. $[\![P]\!](v)$ is an unconditional probability over $\mathcal{V}_\bot$, and the mass which does not reach $\mathcal{V}$ is not normalized away. (In any event, normalization is not a particularly attractive option, as the normalizing constant $\sum_{v \in \mathcal{V}} [\![P]\!](v)$ is not computable in general.)

An important consequence is that expressions can discard such unbounded subcomputations without affecting the likelihood of the containing value. In other words, we can marginalize over discarded subcomputations "for free," since their results are never forced or observed. This is exactly the motivation for using lazy evaluation in prior literature [8, 12, 31, 41, 46, 50]. This allows us to compute the likelihood of a finite observation from a program such as take(3, gen()), even though gen places zero mass on complete ($\bot$-free) finite lists. Because Theorem B.2 holds for all approximants, it is also useful for bounded unrolling: beyond a certain bound, we need not unroll the infinite tails of gen(); we need only meet the observation's demand.

Together, the OBC property and the type system that enforces it provide a straightforward knowledge-compilation mechanism for productive recursive programs under our semantics. The first step toward an inference algorithm is to show that the likelihood at each step along an observation is a sum over a finite set of demands.

Theorem B.3 (Finite demand). *Given an expression e, a variable context ρ containing all free variables in e, an approximant function context ϕ_k, a variable x, and a demand δ, define the function*

$$p_\delta(d) = [\![e]\!]_{\rho[x\mapsto d],\phi_k}(\delta).$$

Then, for each $\delta' \in \mathcal{V}_\perp$, there are finitely many demands $\delta_1, \ldots, \delta_m$ and weights $w_1, \ldots w_m$ such that

$$p_\delta(\delta') = \sum_j w_j \cdot [\delta_j \sqsubseteq \delta'].$$

The demands δ_j are pairwise incompatible, meaning that no pair shares an upper bound; therefore, their upward closures are pairwise disjoint. The only elimination form in Tali is the case expression, and by T-Case, each case expression exhaustively spans a finite set of distinct constructors. Thus the demand placed upon a scrutinee e is generated by a finite decision tree, in which each internal node forces one layer of e and branches on the forced constructor. Two leaves then differ in the constructor at the first position at which their paths diverge. Demands which are merely *incomparable*, but not incompatible, cannot result from this forcing. For example, two such demands might be Cons($\perp$, Nil) and Cons(0, $\perp$), which are pointwise incomparable (the orders on the two subterms are opposites of one another), but nonetheless share an upper bound Cons(0, Nil). The finite constructor domain of scrutinized values then bounds the tree's branching.

Likewise, restricting the function context to an approximant ϕ_k is necessary, not merely cosmetic. Under ϕ_k, recursion is unrolled k times, which bounds the depth of the decision tree. If instead we used the fixed point ϕ, a function such as

$$\text{define consume}(xs : \text{List}) : \text{Unit} := \text{case } xs \text{ of } \{\text{Cons}(hd, tl) \to \text{consume}(tl)\}\{\text{Nil} \to \text{Unit}\}$$

would place infinite demand $\{\text{Nil}, \text{Cons}(\perp, \text{Nil}), \text{Cons}(\perp, \text{Cons}(\perp, \text{Nil})) \ldots\}$ upon its argument xs and the decision tree would have infinite height. (In any case, such functions are not well-typed due to their unguarded recursion, and Theorem 4.5 can be interpreted as a stabilization guarantee on the demand of a function argument as k grows for well-typed programs.[2])

For binding forms such as let , the equations become finite sums when expressed in terms of demands rather than point masses. For example,

$$[\![\text{let } x = e_1 \text{ in } e_2]\!]_{\rho,\phi_k}(\delta) = \sum_{j=1}^{m} c_j \cdot [\![e_1]\!]_{\rho,\phi_k}(\delta_j) + c_0 \left(1 - \sum_{j=1}^{m} [\![e_1]\!]_{\rho,\phi_k}(\delta_j)\right),$$

where m, c_0, $c_1, \ldots, c_m$, and $\delta_1, \ldots, \delta_m$ are obtained by applying Theorem B.3 to e_2. Each δ_j is a demand placed on x by e_2. The same construction reduces all sums over $\mathcal{V}_\perp$ in Figure 3 to finite sums over demands, yielding the probabilistic inference procedure. Given a program P and observation $v \in \mathcal{V}$, we compute the likelihood $[\![P]\!](v)$ by propagating demands "outside-in," forcing each subcomputation only as much as needed to evaluate each step's likelihood. We implement this procedure by incrementally building an arithmetic circuit for the likelihood of the demand at each forced step. The finite set of demands and bounded depth of function unrolling ensure that this circuit is finite.

C TYPE RULES FOR THE TALI LANGUAGE

The complete set of typing rules for Tali is shown in Figure 8. The additional rules not found in Figure 4 are T-Var, T-Flip, T-Geom, T-Constr-Null, T-Let, T-Prog-Main, and T-Defn.

[2]While the example is rejected by the type system, the generality of ϕ_k is still needed to avoid this infinite demand; the inference procedure below builds its circuit at a finite approximant, and the formulation given here is what guarantees that circuit is finite.

$$\textsc{T-Var}\ \frac{\Gamma(x) = \tau}{\Sigma; \Gamma; f \vdash x : \tau} \qquad \textsc{T-Flip}\ \frac{}{\Sigma; \Gamma; f \vdash \mathsf{bern}\ p : \mathsf{Bool}} \qquad \textsc{T-Geom}\ \frac{}{\Sigma; \Gamma; f \vdash \mathsf{geom}\ p : \mathsf{Int}}$$

$$\textsc{T-Later-Sub}\ \frac{\Sigma; \Gamma; f \vdash e : \triangleright^m A \qquad m \leq n}{\Sigma; \Gamma; f \vdash e : \triangleright^n A} \qquad \textsc{T-Constr-Null}\ \frac{C \in C_A \qquad |\mathrm{sig}(C)| = 0}{\Sigma; \Gamma; f \vdash C : A}$$

$$\textsc{T-Constr}\ \frac{C \in C_A \qquad \mathrm{sig}(C) = (A_1, \ldots, A_k) \qquad \forall i \leq k.\ \Sigma; \Gamma; f \vdash e_i : \triangleright^{n+1} A_i}{\Sigma; \Gamma; f \vdash C(\overline{e}) : \triangleright^n A}$$

$$\textsc{T-Let}\ \frac{\Sigma; \Gamma; f \vdash e : \tau \qquad \Sigma; \Gamma, x : \tau; f \vdash e' : \tau'}{\Sigma; \Gamma; f \vdash \mathsf{let}\ x = e\ \mathsf{in}\ e' : \tau'}$$

$$\textsc{T-Call}\ \frac{\begin{array}{c} j \geq 1 \text{ if } g = f \text{ else } j \geq 0 \\ \Sigma(g) = (\triangleright^{m_1} A_1, \ldots, \triangleright^{m_k} A_k) \to \triangleright^n A' \qquad \forall i \leq k.\ \Sigma; \Gamma; f \vdash e_i : \triangleright^{m_i + j} A_i \end{array}}{\Sigma; \Gamma; f \vdash g(\overline{e}) : \triangleright^{n+j} A'}$$

$$\textsc{T-Case}\ \frac{\begin{array}{c} \Sigma; \Gamma; f \vdash e : \triangleright^n A \qquad C_A = \{C_1, \ldots, C_k\} \qquad \forall i \leq k.\ \mathrm{sig}(C_i) = (A_{i1}, \ldots, A_{ik_i}) \\ n \leq m \qquad \forall i \leq k.\ \Sigma; \Gamma, x_{i1} : \triangleright^{n+1} A_{i1}, \ldots, x_{ik_i} : \triangleright^{n+1} A_{ik_i}; f \vdash e_i : \triangleright^m A' \end{array}}{\Sigma; \Gamma; f \vdash \mathsf{case}\ e\ \mathsf{of}\ \overline{\{C_i(\overline{x_i}) \to e_i\}}^k : \triangleright^m A'} \qquad \textsc{T-Prog-Main}\ \frac{\Sigma; \varnothing; \cdot \vdash e : \tau}{\Sigma \vdash e : \tau}$$

$$\textsc{T-Defn}\ \frac{f \notin \mathrm{dom}(\Sigma) \qquad \Sigma' = \Sigma, f : \overline{\tau} \to \tau' \qquad \Sigma'; \overline{x : \tau};\ f \vdash e : \tau' \qquad \Sigma' \vdash P : \tau''}{\Sigma \vdash \mathsf{define}\ f(\overline{x : \tau}) : \tau' = e;\ P : \tau''}$$

Fig. 8. Typing rules for the Tali language.

An important detail of the type system is that function arguments and return values may have arbitrary modal indices in function signatures. This is necessary because arguments may be used freely in the function body: they may be destructured in case expressions, wrapped in constructors, or passed to downstream (possibly recursive) function calls. Without tracking the modalities of arguments and return values, a value at a higher modality could therefore be laundered into a lower one.

Modalities on function argument and return types are not polymorphic. Since Tali is first-order, a simple shift lemma suffices to show that the soundness invariant is monotone in the modal index.

Lemma C.1 (Shift lemma). *Suppose* $\Sigma; \Gamma; f \vdash e : \triangleright^n A$ *and* $j \geq 0$. *Then* $\Sigma; \triangleright^j \Gamma; f \vdash e : \triangleright^{n+j} A$, *where* $\triangleright^j \Gamma$ *is* Γ *where each variable in* Γ *has had its index incremented by* j.

The proof is by induction on the typing derivation.

Bodies:	B	::=	$K(C_1, \ldots, C_m) \mid \text{case } x \text{ of } \overline{\{D(\overline{y}) \to K_D\}}$
Cascades:	$K(C_1)$	::=	$A(C_1)$
	$K(C_1, \ldots, C_m)$		$\text{if bern } \pi \text{ then } A(C_1) \text{ else } K(C_2, \ldots, C_m)$
Arms:	$A(C)$	::=	$\text{let } \overline{w} = \overline{I} \text{ in } C(\overline{F})$
Fields:	F	::=	$f(\overline{a}) \mid w \mid I \mid \text{bern } \pi \mid g(\overline{a}) \mid L(L_1, \ldots, L_r)$
Labels:	$L(L_1)$	::=	L_1
	$L(L_1, \ldots, L_r)$		$\text{if bern } \pi \text{ then } L_1 \text{ else } L(L_2, \ldots, L_r)$
Integer forms:	I	::=	$\text{geom } \pi \mid \min(\text{geom } \pi, h) \mid \min(l + \text{geom } \pi, h)$
			$\mid s + \text{geom } \pi \mid \min(s + \text{geom } \pi, h)$
			$\mid \text{if bern } \pi \text{ then } c \text{ else geom } \pi'$
Call arguments:	a	::=	$y \mid t \mid x \mid x + 1 \mid x - 1$

Fig. 9. Grammar of covering skeletons for a hole of return type T in function f. Productions for fields F and arguments a are selected by type: a field of type T is a self-call $f(\overline{a})$; an Int field is an integer form I, or a let -bound variable w holding one; a Bool field is bern π; for a field of any other type S, the grammar uses a call $g(\overline{a})$ to a generator g for S when one is in scope. Otherwise, if S is an enumeration $L_1 \mid \cdots \mid L_r$, it uses a label cascade $L(L_1, \ldots, L_r)$. We write $\min(e, h)$ for $\text{let } v = e \text{ in if } v \le h \text{ then } v \text{ else } h$.

D EXPERIMENTAL SETUP

D.1 Synthesis DSL grammars

Here we give the family of *covering skeletons* from which the initial program search of Section 5.1.2 draws its starting programs. Let f be the function whose body is the hole, let T be its return type, and let $C_1, \ldots, C_m$ be the constructors of T that occur in the training observations, listed in declaration order. Figure 9 gives the grammar, and Figure 10 shows two skeletons it generates. Each π in the grammar denotes a fresh symbolic parameter, so distinct probabilistic choices in a skeleton have distinct parameters.

Cascades. The body of a basic skeleton is a *cascade*: a right-nested chain of if expressions with one arm for each observed constructor of T. Each guard is a bare bern π, never a test on the function's arguments. A guard based on the arguments could make an arm unreachable and remove its constructor from the support. A cascade over m constructors therefore makes $m - 1$ choices and gives every observed constructor nonzero probability at every call. Together with self-calls in every field of type T, this makes the skeleton covering: every observation built from the observed constructors has nonzero likelihood.

If f has an argument x of algebraic type, the body instead destructures x and places a separate cascade K_D in each arm $D(\overline{y})$. When x is inductive, each self-call passes a destructured field y of the same type, so recursion descends. An arm that binds no such field is a base case, and its cascade contains only observed constructors without fields of type T. When x is not inductive (for example, the tag introduced by call-site splitting in Section 5.1.2), every arm retains all observed constructors, and each tag constructor receives its own independently parameterized cascade.

Fields. Non-integer fields are filled as described in Figure 9. Label cascades for enumerations range over *all* constructors, not only those observed in training, just as geom π covers all of $\mathbb{N}$ rather than only the observed integers. Thus, a label that appears only in held-out data still receives nonzero probability, and parameter fitting removes the mass assigned to unused labels.

Each integer field takes one of the forms I. In these forms, h and l range over integer variables in scope that were not drawn in the current arm (typically the integer parameters of f), s ranges

(a)
```
define genExpr(h : Int) : Expr =
  if bern π1 then Lit(min(geom π2, h))
  else if bern π3 then Var(if bern π4 then X else if bern π5 then Y else Z)
  else Add(genExpr(h), genExpr(h))
```

(b)
```
define sorted(l : Int, h : Int) : List =
  if bern π1 then Nil
  else let x = min(l + geom π2, h) in Cons(x, sorted(x, h))
```

Fig. 10. Example covering skeletons. (a) A cascade over the observed constructors Lit, Var, and Add of Expr. The Var field has the enumeration type X | Y | Z and is filled by a label cascade, and both Add fields are self-calls. (b) The sorted-list skeleton of Section 5.1.2. It differs from the default skeleton if bern π_1 then Nil else Cons(geom π_2, sorted(l, h)) at two choice points: the integer form of the head, and the first argument of the self-call, which threads the bound variable x.

over integer fields of the same constructor that were bound earlier in the arm, and c is the most common field value in the training set. The forms s + geom π and min(s + geom π, h) let one field depend on another. For example, the ordered pair let w_1 = geom π_1 in Pair(w_1, w_1 + geom π_2) is generated this way. The literal-or-draw form if bern π then c else geom π' mixes a point mass at c with a geometric draw. It fits fields whose values concentrate at a mode far from zero more compactly than a geometric draw alone, while preserving the field's full support. Integer fields are let -bound so that later fields and call arguments may refer to them; a binding referenced only by its own field is inlined.

Call arguments. An argument of the inductive type being destructured is the corresponding field y, and a tag argument is the tag t determined by the call site. An integer argument is x, $x + 1$, or $x - 1$, where x is an integer variable in scope, including a field bound in the current arm. This is what allows a drawn value to be threaded into the recursion.

Search space. The *default* skeleton chooses geom π for every integer field and passes each integer parameter of f through unchanged at every self-call. The initial program search enumerates (or performs coordinate descent over) the product of the integer forms and integer arguments at each choice point. Because the options at each choice point depend only on the skeleton, not on choices made elsewhere, this space is fixed before the search begins.

D.2 Transformer setup

D.2.1 Tokenization. Training samples in our datasets are labeled trees. Because constructors have known arities, labeled trees correspond bijectively to sequences of labels in preorder. We serialize each tree with a preorder traversal and assign each label a unique token in the transformer's vocabulary. For trees with integer labels, we instantiate a token for each integer between the minimum and maximum labels in the dataset.

D.2.2 Pretraining. Each pretrained transformer is pretrained on a corpus of 10,000 samples from the corpus generator under test (i.e. program synthesis or random ADT). The program synthesis pretraining corpus generator is trained on n samples, varied during testing. Following pretraining, the transformer is fine-tuned on the original n training samples regardless of pretraining method.

D.2.3 Hyperparameters. All transformer models in this paper use the PyTorch `nn.Transformer` implementation. Each transformer has two hidden layers of size 256, eight attention heads, and a

maximum context window of 1,000 tokens. We train the pretrained transformers using Adam [30] with a batch size of 32 and a constant learning rate of 10^{-3}. We use five pretraining epochs and up to 20 fine-tuning epochs, with early stopping on a validation set (patience 3, minimum delta 10^{-4}).

The full-pool transformers have the same architecture but use a cosine-decay schedule with 1,000 linear warmup steps. We clip gradients at an L^2 norm of 1 and use a dropout rate of 0.1 during training. The full-pool transformers are trained for up to 30 epochs using the same early stopping setup.

E EXPERIMENTAL RESULTS

This appendix gives the numbers behind the figures of Section 6. Real-data tables report the nine real-world cells: SPARQL, and the regex dataset in two character encodings crossed with four training subsets. The *exact* encoding keeps every literal character, while *kind* keeps only the class of letters and digits; the *random* subset is a uniform sample of the corpus, and *repeat*, *range* and *backref* are drawn from regexes containing bounded repeats, set ranges and backreferences, respectively. As in Section 6.2, NLLs are in nats per tree, excesses and differences are medians across seeds, and rates and recalls are means across seeds.

E.1 Synthesized program fit

Table 3 gives the data behind Figure 5: for each real-world cell and training-set size, the program's paired excess NLL over the matched-n untrained transformer, beside the program's support recall. Because a median excess can hide a heavy tail of badly fit trees, the last column also reports CVaR_{10}, the mean paired excess over the worst 10% of covered test trees, at $n = 300$. Table 4 gives the absolute NLLs at $n = 300$, alongside the reference models.

	paired excess, by n				recall, by n				CVaR_{10}
Cell	10	30	100	300	10	30	100	300	300
SPARQL	−10.95	**−23.07**	**−20.94**	**−6.05**	.623	.864	.974	.997	22.3
exact/repeat	2.38	**−6.34**	**−6.57**	**−1.13**	.635	.902	.981	.998	23.9
exact/range	11.21	−2.43	**−3.44**	0.22	.648	.927	.983	.998	26.7
exact/backref	21.85	5.87	−0.80	5.02	.700	.930	.997	1.000	31.3
exact/random	5.04	1.81	0.30	2.22	.600	.896	.985	.998	21.5
kind/repeat	−2.90	**−6.21**	**−5.01**	−0.58	.624	.901	.979	.999	22.8
kind/range	7.36	−3.31	**−3.85**	−0.24	.703	.912	.983	.998	22.9
kind/backref	19.83	6.43	−0.39	4.60	.689	.915	.992	1.000	30.1
kind/random	5.50	4.01	−1.47	0.46	.601	.888	.990	.998	17.9

Table 3. Fit of synthesized programs on real data. *Paired excess:* median, across seeds, of the program's mean excess NLL over the untrained transformer trained on the same n trees, on the test trees the program covers (negative favors the program); bold where the program wins with sign-test $p < 0.01$. *Recall:* mean fraction of test trees to which the program assigns nonzero probability. *$CVaR_{10}$:* median, across seeds, of the mean excess over the worst 10% of covered test trees, at $n = 300$.

Table 5 ablates the synthesis procedure of Section 5 on real data. It builds up from the initial program search alone (*seed*), adding stochastic search (*chain*) and call-site parent annotation (*annot.*), to the full method, which adds the validation-set selection step (*guard*). Parent annotation is the largest single effect, and its sign depends on n: at $n = 300$ it removes 16–28 nats of excess on every cell, but at $n = 10$ it costs fit on every cell. The chain recovers a few nats of that cost at $n = 10$ and has little effect at $n = 300$; the guard changes almost nothing on real data.

Covered trees Cell	Program	No pretraining
SPARQL	73.9	79.7
exact/repeat	70.7	72.2
exact/range	66.2	66.2
exact/backref	87.0	81.7
exact/random	67.1	64.6
kind/repeat	62.2	62.8
kind/range	57.0	57.1
kind/backref	74.6	70.5
kind/random	49.1	48.5

All trees, matched n Cell	No pretraining	Random-pretrained	Program-pretrained
SPARQL	79.7	70.3	56.3
exact/repeat	72.2	74.3	60.1
exact/range	66.2	67.6	57.1
exact/backref	81.7	82.6	74.0
exact/random	64.7	64.6	59.9
kind/repeat	62.9	65.0	52.0
kind/range	57.1	57.5	48.6
kind/backref	70.5	71.4	62.2
kind/random	48.5	48.5	42.7

All trees, references Cell	PTA (smoothed)	Full pool	Full pool (2-layer)
SPARQL	175.4	23.3	26.2
exact/repeat	138.2	–	–
exact/range	134.6	–	–
exact/backref	156.3	–	–
exact/random	115.6	29.5	30.8
kind/repeat	126.9	–	–
kind/range	123.5	–	–
kind/backref	140.6	–	–
kind/random	96.1	24.2	25.0

Table 4. Median test NLL (nats per tree) on real data at $n = 300$. *Covered trees:* the synthesized program and the untrained transformer, both on the test trees the program covers. *All trees, matched n:* the three transformers trained on the same 300 trees. *All trees, references:* a smoothed probabilistic tree automaton (PTA) fit to the same 300 trees, and the full-pool transformer in its six-layer (used in the figures) and two-layer configurations, which were trained only for SPARQL and the *random* regex cells.

E.2 Structure recovery

Table 6 breaks the grid's novel-sample violation rates in Table 2 down by cell: each entry gives the range of per-cell rates within a block, rather than the block mean. It also adds the seed-only program, which stops after the initial program search. The seed-only program recovers the V2 invariants exactly on some cells at $n = 300$, but nearly every novel sample it produces on V1 violates the invariant.

Table 7 reports how often each model's samples are valid in the real-world domains, at the smallest and largest n. Besides the method, it includes two synthesis variants: the seed search with parent annotation but without stochastic search, and the full search without parent annotation. The unannotated program compiles slightly more often as a regex, but rarely produces a valid SPARQL query, since the root query pattern is then drawn from the same rule as every nested one. The last column counts transformer samples that do not decode to a tree at all; programs produce well-typed trees by construction.

Table 8 gives the per-rule violation rates plotted in Figure 6.

Cell	seed	seed +chain	seed +annot.	seed+annot. +chain	seed+annot. +chain+guard
$n = 10$					
SPARQL	−13.7 (.65)	−13.7 (.64)	−7.2 (.65)	−11.0 (.62)	−11.0 (.62)
exact/repeat	−19.1 (.66)	−16.6 (.62)	6.6 (.66)	2.4 (.63)	2.4 (.63)
exact/range	2.1 (.68)	1.7 (.66)	15.9 (.68)	11.2 (.65)	11.2 (.65)
exact/backref	−3.8 (.73)	−2.7 (.66)	24.9 (.73)	21.9 (.70)	21.9 (.70)
exact/random	−0.8 (.63)	0.0 (.61)	6.7 (.63)	5.0 (.60)	5.0 (.60)
kind/repeat	−14.7 (.65)	−13.5 (.61)	−0.6 (.65)	−2.9 (.62)	−2.9 (.62)
kind/range	−1.0 (.73)	−1.1 (.68)	8.6 (.73)	7.4 (.70)	7.4 (.70)
kind/backref	−4.4 (.71)	−1.0 (.66)	22.8 (.71)	19.8 (.69)	19.8 (.69)
kind/random	1.1 (.62)	1.6 (.60)	5.4 (.62)	5.5 (.60)	5.5 (.60)
$n = 300$					
SPARQL	22.0 (1)	21.5 (1)	−5.9 (1)	−6.1 (1)	−6.0 (1)
exact/repeat	16.3 (1)	16.4 (1)	−0.4 (1)	−1.1 (1)	−1.1 (1)
exact/range	19.8 (1)	19.8 (1)	0.3 (1)	0.2 (1)	0.2 (1)
exact/backref	27.1 (1)	26.9 (1)	5.0 (1)	5.0 (1)	5.0 (1)
exact/random	18.5 (1)	18.5 (1)	2.3 (1)	2.2 (1)	2.2 (1)
kind/repeat	16.1 (1)	15.8 (1)	−0.1 (1)	−0.6 (1)	−0.6 (1)
kind/range	19.7 (1)	19.7 (1)	0.0 (1)	−0.2 (1)	−0.2 (1)
kind/backref	26.6 (1)	26.5 (1)	4.7 (1)	4.6 (1)	4.6 (1)
kind/random	18.4 (1)	18.4 (1)	0.5 (1)	0.5 (1)	0.5 (1)

Table 5. Ablation of the synthesis procedure at $n = 10$ and $n = 300$: median paired excess NLL over the untrained transformer on the test trees each program covers (negative favors the program), with mean support recall in parentheses. *seed*: initial program search only; *+chain*: followed by stochastic search; *+annot.*: with call-site parent annotation; *+guard*: with validation-set selection among the top-k programs. The rightmost column is the method evaluated in Section 6.

	No pretraining		Random-pretrained		Program		Seed only	
Block	$n = 10$	$n = 300$	$n = 10$	$n = 300$	$n = 10$	$n = 300$	$n = 10$	$n = 300$
S1–S3	.22–.41	.01–.13	.00–.07	.00–.06	.02–.06	.00–.04	.01–.06	.00–.10
S4	.37–.49	.09–.28	.14–.40	.09–.45	.12–.28	.07–.39	.17–.28	.28–.42
V1	.39–.76	.26–.35	.47–.73	.26–.36	.38–.73	.82–.90	.46–.78	.98–.99
V2	.33–.65	.18–.46	.32–.39	.21–.54	.12–.18	.13–.30	.13–.14	.00–.30
V3	.33–.99	.11–.87	.21–.98	.64–.99	.29–.99	.71–1.00	.27–.99	.72–1.00

Table 6. Mean rate at which novel, well-formed samples fall outside the ground truth's support on the synthetic grid, per block, arm and training-set size (lower is better). Where a block has several cells, the entry gives the range of the per-cell means. *Program* is the synthesized program, as in Table 2; *Seed only* is the program produced by the initial program search alone.

E.3 Data augmentation

Table 9 gives the data behind Figure 7. The first two blocks compare the program-pretrained transformer with each control trained on the same n trees, and the last gives its own test NLL beside the full-pool reference. Every transformer covers every test tree, so these comparisons are over all test trees and carry no recall caveat.

	Regex		SPARQL					
	compiles		valid		well-formed		malformed	
Arm	$n = 10$	$n = 300$	$n = 10$	$n = 300$	$n = 10$	$n = 300$	$n = 10$	$n = 300$
Program	.647	.609	.480	.410	.608	.564	–	–
Seed + annot./lit.	.647	.612	.466	.421	.592	.584	–	–
Unannotated	.688	.693	.028	.016	.034	.018	–	–
Program-pretrained	.476	.460	.355	.450	.431	.545	.392	.344
Random-pretrained	.205	.121	.027	.211	.030	.252	.763	.633
No pretraining	.003	.042	.000	.022	.000	.023	.978	.900
Full pool	.996		.966		.989		.009	
Full pool (2-layer)	.986		.943		.976		.021	
Pool	–		.981		1.000		–	

Table 7. Fraction of samples that are valid in the real-world domains, at $n = 10$ and $n = 300$ (mean across seeds). *Regex compiles:* accepted by Python's `re.compile`. *SPARQL valid:* well-formed and satisfying the four scoping rules of Figure 6; *well-formed:* printable as a query; *malformed:* a transformer sample whose token sequence does not decode to a tree. *Program*, *Seed + annot./lit.* and *Unannotated* are synthesized programs; the full-pool transformers are trained once, so a single entry spans both n. *Pool:* the SPARQL training data itself.

Regex	local			agreement
Arm	no compile	repeat	range	backref.
Program	.494	.005	.005	.767
Program-pretrained	.500	.025	.002	.246
Random-pretrained	.792	.307	.006	.388
No pretraining	.949	.093	.000	.288
Full pool	.005	.002	.002	.062

SPARQL	local		agreement			
Arm	W	S1	S2	S3	S4	N1
Program	.436	.060	.893	.309	.652	.986
Program-pretrained	.455	.030	.897	.128	.626	.550
Random-pretrained	.748	.082	.919	.087	.509	.482
No pretraining	.977	.000	.806	.000	.260	.184
Full pool	.011	.004	.030	.024	.121	.005
Pool	.000	.007	.005	.000	.106	.000

Table 8. Fraction of samples violating each validity rule at $n = 300$ (mean across seeds; lower is better), as in Figure 6. *Regex:* fails `re.compile`; repeat `{m,n}` with $m > n$; set range with reversed bounds; backreference to an unclosed group. *SPARQL:* not printable as a query (W); misplaced or nested aggregate (S1); aggregate query projects a non-`GROUP BY` variable (S2); `BIND` to an in-scope variable (S3); `AS ?v` rebinds a bound variable (S4); variables not numbered in first-use order (N1). Except for compilation and W, rates are among samples containing the rule's construct. *Pool:* the SPARQL training data, which includes some queries the specification forbids.

Cell	$n = 10$	$n = 30$	$n = 100$	$n = 300$	
Δ *NLL vs. random-pretrained (fraction of seeds won)*					
SPARQL	−19.9 .93	−28.7 1	−20.7 1	−14.3 1	
exact/repeat	−18.3 1	−22.8 1	−17.2 1	−14.2 1	
exact/range	−12.0 .96	−17.6 1	−12.2 1	−10.3 1	
exact/backref	−10.1 .89	−14.0 1	−9.3 1	−8.8 1	
exact/random	2.7 .44	−8.6 .96	−5.7 1	−5.1 1	
kind/repeat	−17.6 .93	−21.9 1	−15.4 1	−13.3 1	
kind/range	−14.4 .96	−16.7 1	−11.5 1	−9.4 1	
kind/backref	−8.7 .85	−14.0 1	−9.9 1	−9.4 1	
kind/random	−1.6 .56	−9.2 1	−6.3 1	−6.0 1	
Δ *NLL vs. no pretraining (fraction of seeds won)*					
SPARQL	−35.4 1	−45.5 1	−38.4 1	−23.6 1	
exact/repeat	−25.9 1	−21.7 1	−15.5 1	−11.8 1	
exact/range	−16.4 1	−17.4 1	−12.1 1	−9.1 1	
exact/backref	−13.5 1	−12.4 1	−9.2 1	−7.7 1	
exact/random	−5.1 .70	−11.2 1	−6.5 1	−5.1 1	
kind/repeat	−22.6 .96	−19.6 1	−15.1 1	−10.9 1	
kind/range	−16.9 1	−16.4 1	−12.2 1	−8.7 1	
kind/backref	−12.5 .93	−12.0 1	−9.8 1	−8.4 1	
kind/random	−6.5 .59	−9.8 1	−7.0 1	−5.8 1	
Student test NLL					full pool
SPARQL	103.2	75.2	63.1	56.3	23.3
exact/repeat	97.5	74.4	66.6	60.1	–
exact/range	86.3	69.4	62.3	57.1	–
exact/backref	112.7	91.8	83.0	74.0	–
exact/random	92.1	71.4	64.6	59.9	29.5
kind/repeat	85.5	65.9	57.8	52.0	–
kind/range	75.3	60.0	53.2	48.6	–
kind/backref	98.7	79.9	70.1	62.2	–
kind/random	74.4	55.0	47.4	42.7	24.2

Table 9. Pretraining on samples from a synthesized program. Δ *NLL:* median, across seeds, of the program-pretrained transformer's test NLL minus that of the random-pretrained or untrained transformer trained on the same n trees (negative favors program pretraining), with the fraction of seeds on which program pretraining wins in small type. *Student test NLL:* median test NLL of the program-pretrained transformer, beside the full-pool transformer, which was trained only for SPARQL and the *random* regex cells.